

深度学习所需的矩阵微积分

Terence Parr and Jeremy Howard

2026 年 8 月 19 日

(我们在旧金山大学的数据科学硕士项目中授课，还有一些其他“不务正业”的项目正在进行中。你可能知道 Terence 是 [ANTLR 解析器生成器](#) 的创造者。如需更多资料，请参阅 Jeremy 的 [fast.ai 课程](#)，以及旧金山大学数据研究所的[深度学习课程线下版](#)。)

[HTML 版本](#) (PDF 和 HTML 都是使用 [bookish](#) 从标记源生成的)

摘要

本文试图解释理解深度神经网络训练所需的全部矩阵微积分知识。我们假设读者除了微积分 1 中学到的知识外没有其他数学基础，并在需要之处提供链接帮助您复习必要的数学。请注意，在开始学习如何训练并在实践中使用深度学习之前，您并不需要理解这些内容；相反，这些内容是为那些已经熟悉神经网络基础知识、并希望加深对底层数学理解的人准备的。如果在阅读途中某个地方卡住了，别担心——只需回头重读前一节，并尝试写下并动手演算一些例子。如果仍然卡住，我们很乐意在 [forums.fast.ai 的 Theory 版块](#) 回答您的问题。**注意：**论文末尾有一个[参考文献部分](#)，总结了本文讨论的所有关键矩阵微积分规则和术语。

目录

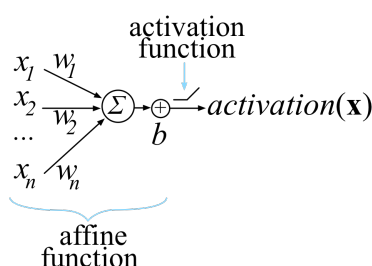
1 引言	3
2 回顾：标量导数规则	4
3 向量微积分与偏导数导论	5
4 矩阵微积分	6
4.1 雅可比矩阵的推广	7
4.2 向量逐元素二元算子的导数	9
4.3 涉及标量扩展的导数	12
4.4 向量求和归约	13
4.5 链式法则	14
4.5.1 单变量链式法则	14
4.5.2 单变量全导数链式法则	18
4.5.3 向量链式法则	21
5 神经元激活的梯度	23
6 神经网络损失函数的梯度	26
6.1 对权重的梯度	26
6.2 对偏置的导数	28
7 总结	29
8 矩阵微积分参考	30
8.1 梯度与雅可比矩阵	30
8.2 向量上的逐元素运算	30
8.3 标量扩展	31
8.4 向量归约	31
8.5 链式法则	32
9 记号	32
10 资源	33

1 引言

我们大多数人最后一次接触微积分是在学校里，但导数是机器学习的关键组成部分，尤其是深度神经网络——它通过优化损失函数来训练。随便拿起一篇机器学习论文或某个库（如 [PyTorch](#)）的文档，微积分就会像节日里的远房亲戚一样，呼啸着闯回你的生活。而且这可不是什么老掉牙的标量微积分——你需要的是微分矩阵微积分，它是[线性代数](#)与[多元微积分](#)的”闪婚”产物。

好吧……也许需要这个词并不恰当；Jeremy 的课程展示了如何仅凭最低限度的标量微积分就能成为世界一流的深度学习实践者，这要归功于利用现代深度学习库内置的自动微分。但如果你真的想理解这些库的内部运作机制，并真正读懂那些讨论模型训练技术最新进展的学术论文，你就需要理解矩阵微积分领域的某些内容。

例如，神经网络中单个计算单元的激活通常通过边的权重向量 $\mathbf{w}$ 与输入向量 $\mathbf{x}$ 的点积（来自线性代数）再加上一个标量偏置（阈值）来计算： $z(\mathbf{x}) = \sum_i^n w_i x_i + b = \mathbf{w} \cdot \mathbf{x} + b$ 。函数 $z(\mathbf{x})$ 被称为该单元的仿射函数，其后接一个[修正线性单元](#)，它会将负值裁剪为零： $\max(0, z(\mathbf{x}))$ 。这样的计算单元有时被称为”人工神经元”，如图所示：



神经网络由许多这样的单元组成，它们被组织成多个称为层的神经元集合。某一层单元的激活会成为下一层单元的输入。最后一层中一个或多个单元的激活被称为网络输出。

训练这个神经元意味着选择权重 $\mathbf{w}$ 和偏置 b ，使得对所有 N 个输入 $\mathbf{x}$ 都能得到期望的输出。为此，我们最小化一个损失函数，它将网络最终的 $activation(\mathbf{x})$ 与所有输入 $\mathbf{x}$ 向量的 $target(\mathbf{x})$ ($\mathbf{x}$ 的期望输出) 进行比较。为了最小化损失，我们使用梯度下降的某种变体，例如普通的[随机梯度下降](#) (SGD)、带动量的 SGD，或 [Adam](#)。所有这些方法都需要 $activation(\mathbf{x})$ 关于模型参数 $\mathbf{w}$ 和 b 的偏导数（梯度）。我们的目标是逐步微调 $\mathbf{w}$ 和 b ，使得整体损失函数在所有 $\mathbf{x}$ 输入上持续变小。

如果我们足够细心，就可以通过对一个常见损失函数（均方误差）的标量版本求导来推导出梯度：

$$\frac{1}{N} \sum_{\mathbf{x}} (target(\mathbf{x}) - activation(\mathbf{x}))^2 = \frac{1}{N} \sum_{\mathbf{x}} (target(\mathbf{x}) - \max(0, \sum_i^{|x|} w_i x_i + b))^2$$

但这只是一个神经元，而神经网络必须同时训练所有层中所有神经元的权重和偏置。由于存在多个输入和（可能）多个网络输出，我们确实需要函数关于向量的导数的一般规则，甚至需要向量值函数关于向量的导数规则。

本文详细推导了计算关于向量的偏导数的一些重要规则，特别是那些对训练神经网络有用的规则。这一领域被称为矩阵微积分，好消息是，我们只需要该领域的一小部分，我们将在本文中介绍这部分内容。虽然网上有大量关于多元微积分和线性代数的资料，但这两门课通常作为两门独立的本科课程来教授，因此大多数资料都将二者分开处理。那些确实讨论矩阵微积分的页面，往往只是带有极少解释的规则列表，或者只是整个故事的一部分。而且由于它们使用密集的记号、对基础概念的讨论极少，除了少数数学家读者之外，对其他人来说往往相当晦涩。（参见文末带注释的资源列表。）

与此相反，我们将重新推导并重新发现一些关键的矩阵微积分规则，以努力解释它们。事实证明，矩阵微积分其实并不难！不需要学习几十条新规则；只需要几个关键概念。我们希望这篇短文能让你快速进入矩阵微积分这个与训练神经网络息息相关的世界。我们假设你已经熟悉神经网络架构和训练的基础知识。如果还不熟悉，请前往 [Jeremy 的课程](#) 并完成其中第一部分，完成后我们在这里等你回来。（请注意，与许多更偏学术的方法不同，我们强烈建议先学习如何在实际中训练和使用神经网络，然后再学习底层数学。有了上下文背景，数学会更容易理解；此外，要成为一名高效的实践者，并不需要彻底搞懂所有这些微积分。）

关于记号的说明：Jeremy 的课程完全使用代码而非数学记号来解释概念，因为代码中不熟悉的函数很容易搜索和试验。在本文中，我们反其道而行之：本文包含大量数学记号，因为本文的目标之一就是帮助你理解在深度学习论文和书籍中会看到的记号。在论文的[结尾](#)，你会找到所用记号的简要表格，其中包含可用于搜索更多细节的单词或短语。

2 回顾：标量导数规则

希望你还能记得这些主要标量导数规则中的一部分。如果对此记忆有些模糊，可以看看 [Khan Academy 关于标量导数规则的视频](#)。

规则	$f(x)$	关于 x 的标量导数记号	示例
常数	c	0	$\frac{d}{dx}99 = 0$
常数乘法	cf	$c\frac{df}{dx}$	$\frac{d}{dx}3x = 3$
幂法则	x^n	nx^{n-1}	$\frac{d}{dx}x^3 = 3x^2$
和法则	$f + g$	$\frac{df}{dx} + \frac{dg}{dx}$	$\frac{d}{dx}(x^2 + 3x) = 2x + 3$
差法则	$f - g$	$\frac{df}{dx} - \frac{dg}{dx}$	$\frac{d}{dx}(x^2 - 3x) = 2x - 3$
积法则	fg	$f\frac{dg}{dx} + \frac{df}{dx}g$	$\frac{d}{dx}x^2x = x^2 + x2x = 3x^2$
链式法则	$f(g(x))$	$\frac{df(u)}{du}\frac{du}{dx}$, 令 $u = g(x)$	$\frac{d}{dx}\ln(x^2) = \frac{1}{x^2}2x = \frac{2}{x}$

三角函数、指数函数等还有其他规则，你可以在 [Khan Academy 微分学课程](#) 中找到。

当一个函数只有一个参数 $f(x)$ 时，你经常会看到 f' 和 $f'(x)$ 被用作 $\frac{d}{dx}f(x)$ 的简写。我们不建议使用这种记号，因为它没有明确我们是对哪个变量求导。

你可以把 $\frac{d}{dx}$ 看作一个把单参数函数映射到另一个函数的算子。这意味着 $\frac{d}{dx}f(x)$ 把 $f(x)$ 映射到它关于 x 的导数，这与 $\frac{df(x)}{dx}$ 是同一回事。另外，如果 $y = f(x)$ ，那么 $\frac{dy}{dx} = \frac{df(x)}{dx} = \frac{d}{dx}f(x)$ 。把导数看作算子有助于简化复杂的导数，因为该算子满足分配律，可以让我们提取常数。例如，在下面的等式中，我们可以提取常数 9，并把导数算子分配到括号内的各项上。

$$\frac{d}{dx}9(x + x^2) = 9\frac{d}{dx}(x + x^2) = 9\left(\frac{d}{dx}x + \frac{d}{dx}x^2\right) = 9(1 + 2x) = 9 + 18x$$

这个过程把 $9(x + x^2)$ 的导数化简为一些算术运算以及 x 和 x^2 的导数，后者比原来的导数容易求解得多。

3 向量微积分与偏导数导论

神经网络层并不是单一参数 $f(x)$ 的单一函数。因此，让我们转向多参数函数，例如 $f(x, y)$ 。比如， xy （即 x 与 y 的乘积）的导数是什么？换言之，当我们扰动变量时，乘积 xy 会如何变化？这取决于我们改变的是 x 还是 y 。我们一次只对一个变量（参数）求导数，于是对这个双参数函数，我们得到两个不同的偏导数（一个对 x ，一个对 y ）。偏导数算子不用 $\frac{d}{dx}$ ，而是用 $\frac{\partial}{\partial x}$ （一个美术体的 d ，而不是希腊字母 δ ）。因此， $\frac{\partial}{\partial x}xy$ 和 $\frac{\partial}{\partial y}xy$ 是 xy 的偏导数；通常它们被简称为偏导。对于单参数函数，算子 $\frac{\partial}{\partial x}$ 与 $\frac{d}{dx}$ 等价（对于足够光滑的函数）。不过，使用 $\frac{d}{dx}$ 更好，可以清楚地表明你指的是标量导数。

关于 x 的偏导数就是通常的标量导数，只需把方程中任何其他变量都当作常数。考虑函数 $f(x, y) = 3x^2y$ 。关于 x 的偏导数写作 $\frac{\partial}{\partial x}3x^2y$ 。从 $\frac{\partial}{\partial x}$ 的角度看，这里有三个常数：3、2 和 y 。因此， $\frac{\partial}{\partial x}3yx^2 = 3y\frac{\partial}{\partial x}x^2 = 3y2x = 6yx$ 。关于 y 的偏导数则把 x 当作常数： $\frac{\partial}{\partial y}3x^2y = 3x^2\frac{\partial}{\partial y}y = 3x^2\frac{\partial y}{\partial y} = 3x^2 \times 1 = 3x^2$ 。

$3x^2$ 。在继续之前，最好亲自推导一遍这些结果，否则文章的其余部分将难以理解。如果你需要帮助，这里有一个[可汗学院关于偏导的视频](#)。

为了明确我们做的是向量微积分而不仅仅是多元微积分，让我们考虑如何处置我们为 $f(x, y) = 3x^2y$ 计算出的偏导数 $\frac{\partial f(x, y)}{\partial x}$ 和 $\frac{\partial f(x, y)}{\partial y}$ （即 $\frac{\partial}{\partial x} f(x, y)$ 和 $\frac{\partial}{\partial y} f(x, y)$ 的另一种写法）。与其让它们随意散落、毫无组织，不如把它们组织成一个水平向量。我们把这个向量称为 $f(x, y)$ 的梯度，并写成：

$$\nabla f(x, y) = \left[\frac{\partial f(x, y)}{\partial x}, \frac{\partial f(x, y)}{\partial y} \right] = [6yx, 3x^2]$$

因此， $f(x, y)$ 的梯度就是由它的偏导组成的向量。梯度属于向量微积分的范畴，该范畴处理的是把 n 个标量参数映射到单个标量的函数。现在，让我们大胆一点，同时考虑多个函数的导数。

4 矩阵微积分

当我们从单个函数的导数转向多个函数的导数时，我们就从向量微积分的范畴进入矩阵微积分的范畴。让我们计算两个函数的偏导数，这两个函数都接受两个参数。我们可以沿用上一节的 $f(x, y) = 3x^2y$ ，但同时引入 $g(x, y) = 2x + y^8$ 。 g 的梯度有两个分量，每个参数对应一个偏导数：

$$\frac{\partial g(x, y)}{\partial x} = \frac{\partial 2x}{\partial x} + \frac{\partial y^8}{\partial x} = 2 \frac{\partial x}{\partial x} + 0 = 2 \times 1 = 2$$

以及

$$\frac{\partial g(x, y)}{\partial y} = \frac{\partial 2x}{\partial y} + \frac{\partial y^8}{\partial y} = 0 + 8y^7 = 8y^7$$

由此得到梯度 $\nabla g(x, y) = [2, 8y^7]$ 。

梯度向量组织了一个特定标量函数的所有偏导数。如果我们有两个函数，也可以把它们的梯度堆叠起来，组织成一个矩阵。这样做之后，我们就得到雅可比矩阵（或简称雅可比），其中梯度作为行：

$$J = \begin{bmatrix} \nabla f(x, y) \\ \nabla g(x, y) \end{bmatrix} = \begin{bmatrix} \frac{\partial f(x, y)}{\partial x} & \frac{\partial f(x, y)}{\partial y} \\ \frac{\partial g(x, y)}{\partial x} & \frac{\partial g(x, y)}{\partial y} \end{bmatrix} = \begin{bmatrix} 6yx & 3x^2 \\ 2 & 8y^7 \end{bmatrix}$$

欢迎来到矩阵微积分的世界！

请注意，雅可比矩阵有多种表示方式。我们使用的是所谓的[分子布局](#)，但许多论文和软件会使用分母布局。它只是分子布局雅可比矩阵的转置（沿其对角线翻转）：

$$\begin{bmatrix} 6yx & 2 \\ 3x^2 & 8y^7 \end{bmatrix}$$

4.1 雅可比矩阵的推广

到目前为止，我们只看了雅可比矩阵的一个具体例子。为了更一般地定义雅可比矩阵，让我们把多个参数合并为一个向量参数： $f(x, y, z) \Rightarrow f(\mathbf{x})$ 。（在文献中，你有时也会看到用 $\vec{x}$ 表示向量的记号。）小写粗体字母如 $\mathbf{x}$ 表示向量，斜体字母如 x 表示标量。 x_i 是向量 $\mathbf{x}$ 的第 i 个元素，之所以用斜体，是因为向量的单个元素是标量。我们还必须定义向量 $\mathbf{x}$ 的方向。我们默认所有向量都是垂直的，大小为 $n \times 1$ ：

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}$$

对于多个标量值函数，我们可以像处理参数那样，把它们全部合并成一个向量。设 $\mathbf{y} = \mathbf{f}(\mathbf{x})$ 是一个由 m 个标量值函数组成的向量，其中每个函数都接受一个长度为 $n = |\mathbf{x}|$ 的向量 $\mathbf{x}$ ， $|\mathbf{x}|$ 是 $\mathbf{x}$ 中元素的基数（个数）。 $\mathbf{f}$ 中的每个函数 f_i 都返回一个标量，正如上一节那样：

$$\begin{aligned} y_1 &= f_1(\mathbf{x}) \\ y_2 &= f_2(\mathbf{x}) \\ &\vdots \\ y_m &= f_m(\mathbf{x}) \end{aligned}$$

例如，我们可以把上一节的 $f(x, y) = 3x^2y$ 和 $g(x, y) = 2x + y^8$ 表示为

$$\begin{aligned} y_1 &= f_1(\mathbf{x}) = 3x_1^2x_2 && (\text{substituting } x_1 \text{ for } x, x_2 \text{ for } y) \\ y_2 &= f_2(\mathbf{x}) = 2x_1 + x_2^8 \end{aligned}$$

通常情况下 $m = n$ ，因为对于 $\mathbf{x}$ 向量的每个元素，我们都会得到一个标量函数结果。例如，考虑恒等函数 $\mathbf{y} = \mathbf{f}(\mathbf{x}) = \mathbf{x}$ ：

$$\begin{aligned} y_1 &= f_1(\mathbf{x}) = x_1 \\ y_2 &= f_2(\mathbf{x}) = x_2 \\ &\vdots \\ y_n &= f_n(\mathbf{x}) = x_n \end{aligned}$$

在这种情形下，我们有 $m = n$ 个函数和参数。不过一般来说，雅可比矩阵是所有 $m \times n$ 个可能的偏导数（ m 行 n 列）的集合，也就是关于 $\mathbf{x}$ 的 m 个梯度的堆叠：

$$\frac{\partial \mathbf{y}}{\partial \mathbf{x}} = \begin{bmatrix} \nabla f_1(\mathbf{x}) \\ \nabla f_2(\mathbf{x}) \\ \dots \\ \nabla f_m(\mathbf{x}) \end{bmatrix} = \begin{bmatrix} \frac{\partial}{\partial \mathbf{x}} f_1(\mathbf{x}) \\ \frac{\partial}{\partial \mathbf{x}} f_2(\mathbf{x}) \\ \dots \\ \frac{\partial}{\partial \mathbf{x}} f_m(\mathbf{x}) \end{bmatrix} = \begin{bmatrix} \frac{\partial}{\partial x_1} f_1(\mathbf{x}) & \frac{\partial}{\partial x_2} f_1(\mathbf{x}) & \dots & \frac{\partial}{\partial x_n} f_1(\mathbf{x}) \\ \frac{\partial}{\partial x_1} f_2(\mathbf{x}) & \frac{\partial}{\partial x_2} f_2(\mathbf{x}) & \dots & \frac{\partial}{\partial x_n} f_2(\mathbf{x}) \\ \dots & \dots & \dots & \dots \\ \frac{\partial}{\partial x_1} f_m(\mathbf{x}) & \frac{\partial}{\partial x_2} f_m(\mathbf{x}) & \dots & \frac{\partial}{\partial x_n} f_m(\mathbf{x}) \end{bmatrix}$$

每个 $\frac{\partial}{\partial \mathbf{x}} f_i(\mathbf{x})$ 都是一个水平 n 向量，因为偏导数是关于向量 $\mathbf{x}$ 的，而 $\mathbf{x}$ 的长度为 $n = |\mathbf{x}|$ 。如果对 $\mathbf{x}$ 求偏导，雅可比矩阵的宽度就是 n ，因为有 n 个可以扰动的参数，每个参数都可能改变函数的值。因此，对于 m 个方程，雅可比矩阵始终有 m 行。借助图形来思考雅可比矩阵可能的形状会有所帮助：

		向量	
		标量	
		x	$\mathbf{x}$
标量			
f		$\frac{\partial f}{\partial x}$	$\frac{\partial f}{\partial \mathbf{x}}$
向量			
$\mathbf{f}$		$\frac{\partial \mathbf{f}}{\partial x}$	$\frac{\partial \mathbf{f}}{\partial \mathbf{x}}$

恒等函数 $\mathbf{f}(\mathbf{x}) = \mathbf{x}$ （其中 $f_i(\mathbf{x}) = x_i$ ）的雅可比矩阵有 n 个函数，每个函数有 n 个参数，这些参数保存在单个向量 $\mathbf{x}$ 中。因此，由于 $m = n$ ，雅可比矩阵是一个方阵：

$$\begin{aligned}
 \frac{\partial \mathbf{y}}{\partial \mathbf{x}} &= \begin{bmatrix} \frac{\partial}{\partial \mathbf{x}} f_1(\mathbf{x}) \\ \frac{\partial}{\partial \mathbf{x}} f_2(\mathbf{x}) \\ \dots \\ \frac{\partial}{\partial \mathbf{x}} f_m(\mathbf{x}) \end{bmatrix} = \begin{bmatrix} \frac{\partial}{\partial x_1} f_1(\mathbf{x}) & \frac{\partial}{\partial x_2} f_1(\mathbf{x}) & \dots & \frac{\partial}{\partial x_n} f_1(\mathbf{x}) \\ \frac{\partial}{\partial x_1} f_2(\mathbf{x}) & \frac{\partial}{\partial x_2} f_2(\mathbf{x}) & \dots & \frac{\partial}{\partial x_n} f_2(\mathbf{x}) \\ \dots & \dots & \dots & \dots \\ \frac{\partial}{\partial x_1} f_m(\mathbf{x}) & \frac{\partial}{\partial x_2} f_m(\mathbf{x}) & \dots & \frac{\partial}{\partial x_n} f_m(\mathbf{x}) \end{bmatrix} \\
 &= \begin{bmatrix} \frac{\partial}{\partial x_1} x_1 & \frac{\partial}{\partial x_2} x_1 & \dots & \frac{\partial}{\partial x_n} x_1 \\ \frac{\partial}{\partial x_1} x_2 & \frac{\partial}{\partial x_2} x_2 & \dots & \frac{\partial}{\partial x_n} x_2 \\ \dots & \dots & \dots & \dots \\ \frac{\partial}{\partial x_1} x_n & \frac{\partial}{\partial x_2} x_n & \dots & \frac{\partial}{\partial x_n} x_n \end{bmatrix} \\
 &\quad \text{(and since } \frac{\partial}{\partial x_j} x_i = 0 \text{ for } j \neq i) \\
 &= \begin{bmatrix} \frac{\partial}{\partial x_1} x_1 & 0 & \dots & 0 \\ 0 & \frac{\partial}{\partial x_2} x_2 & \dots & 0 \\ \dots & \dots & \ddots & \dots \\ 0 & 0 & \dots & \frac{\partial}{\partial x_n} x_n \end{bmatrix}
 \end{aligned}$$

$$= \begin{bmatrix} 1 & 0 & \dots & 0 \\ 0 & 1 & \dots & 0 \\ & & \ddots & \\ 0 & 0 & \dots & 1 \end{bmatrix}$$

$$= I \quad (I \text{ is the identity matrix with ones down the diagonal})$$

在继续之前，请确保你能独立推导出上面的每一步。如果卡住了，只需孤立地考虑矩阵的每个元素，并应用通常的标量导数规则。这是一个普遍有用的技巧：把向量表达式化简为一组标量表达式，然后求出所有偏导，最后把结果恰当地组合成向量和矩阵。

还要注意区分矩阵是垂直的 $\mathbf{x}$ 还是水平的 $\mathbf{x}^T$ ，其中 $\mathbf{x}^T$ 表示 $\mathbf{x}$ 的转置。同时要留意某个量是标量值函数 $y = \dots$ ，还是函数向量（或向量值函数） $\mathbf{y} = \dots$ 。

4.2 向量逐元素二元算子的导数

向量的逐元素二元运算（例如向量加法 $\mathbf{w} + \mathbf{x}$ ）之所以重要，是因为我们可以把许多常见的向量运算（如向量与标量的乘法）表示为逐元素二元运算。所谓“逐元素二元运算”，我们指的是对每个向量的第一个元素应用某个算子，得到输出的第一个元素，再对输入向量的第二组元素应用该算子，得到输出的第二个元素，依此类推。例如，numpy 或 tensorflow 默认就是这样应用所有基本数学算子的。深度学习中经常出现的例子有 $\max(\mathbf{w}, \mathbf{x})$ 和 $\mathbf{w} > \mathbf{x}$ （返回一个由 1 和 0 组成的向量）。

我们可以用记号 $\mathbf{y} = \mathbf{f}(\mathbf{w}) \circ \mathbf{g}(\mathbf{x})$ 来推广逐元素二元运算，其中 $m = n = |\mathbf{y}| = |\mathbf{w}| = |\mathbf{x}|$ 。（提醒： $|\mathbf{x}|$ 是 $\mathbf{x}$ 中元素的个数。） $\circ$ 符号表示任意逐元素算子（例如 $+$ ），而不是 $\circ$ 函数复合算子。下面就是当我们放大来考察标量方程时，方程 $\mathbf{y} = \mathbf{f}(\mathbf{w}) \circ \mathbf{g}(\mathbf{x})$ 的样子：

$$\begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix} = \begin{bmatrix} f_1(\mathbf{w}) \circ g_1(\mathbf{x}) \\ f_2(\mathbf{w}) \circ g_2(\mathbf{x}) \\ \vdots \\ f_n(\mathbf{w}) \circ g_n(\mathbf{x}) \end{bmatrix}$$

这里我们纵向写出 n （而非 m ）个方程，以强调逐元素运算的结果是大小为 $m = n$ 的向量这一事实。

利用上一节的思想，我们可以看到，关于 $\mathbf{w}$ 的雅可比矩阵的一般情形是如下方阵：

$$J_{\mathbf{w}} = \frac{\partial \mathbf{y}}{\partial \mathbf{w}} = \begin{bmatrix} \frac{\partial}{\partial w_1}(f_1(\mathbf{w}) \circ g_1(\mathbf{x})) & \frac{\partial}{\partial w_2}(f_1(\mathbf{w}) \circ g_1(\mathbf{x})) & \dots & \frac{\partial}{\partial w_n}(f_1(\mathbf{w}) \circ g_1(\mathbf{x})) \\ \frac{\partial}{\partial w_1}(f_2(\mathbf{w}) \circ g_2(\mathbf{x})) & \frac{\partial}{\partial w_2}(f_2(\mathbf{w}) \circ g_2(\mathbf{x})) & \dots & \frac{\partial}{\partial w_n}(f_2(\mathbf{w}) \circ g_2(\mathbf{x})) \\ \dots & \dots & \dots & \dots \\ \frac{\partial}{\partial w_1}(f_n(\mathbf{w}) \circ g_n(\mathbf{x})) & \frac{\partial}{\partial w_2}(f_n(\mathbf{w}) \circ g_n(\mathbf{x})) & \dots & \frac{\partial}{\partial w_n}(f_n(\mathbf{w}) \circ g_n(\mathbf{x})) \end{bmatrix}$$

而关于 $\mathbf{x}$ 的雅可比矩阵为：

$$J_{\mathbf{x}} = \frac{\partial \mathbf{y}}{\partial \mathbf{x}} = \begin{bmatrix} \frac{\partial}{\partial x_1}(f_1(\mathbf{w}) \circ g_1(\mathbf{x})) & \frac{\partial}{\partial x_2}(f_1(\mathbf{w}) \circ g_1(\mathbf{x})) & \dots & \frac{\partial}{\partial x_n}(f_1(\mathbf{w}) \circ g_1(\mathbf{x})) \\ \frac{\partial}{\partial x_1}(f_2(\mathbf{w}) \circ g_2(\mathbf{x})) & \frac{\partial}{\partial x_2}(f_2(\mathbf{w}) \circ g_2(\mathbf{x})) & \dots & \frac{\partial}{\partial x_n}(f_2(\mathbf{w}) \circ g_2(\mathbf{x})) \\ \dots & \dots & \dots & \dots \\ \frac{\partial}{\partial x_1}(f_n(\mathbf{w}) \circ g_n(\mathbf{x})) & \frac{\partial}{\partial x_2}(f_n(\mathbf{w}) \circ g_n(\mathbf{x})) & \dots & \frac{\partial}{\partial x_n}(f_n(\mathbf{w}) \circ g_n(\mathbf{x})) \end{bmatrix}$$

这看起来相当庞杂，但幸运的是，雅可比矩阵往往是对角矩阵，即除对角线外处处为零的矩阵。由于这能极大地简化雅可比矩阵，让我们详细考察一下，对于逐元素运算，雅可比矩阵在什么情况下会化为对角矩阵。

在对角雅可比矩阵中，所有非对角线元素都为零，即当 $j \neq i$ 时 $\frac{\partial}{\partial w_j}(f_i(\mathbf{w}) \circ g_i(\mathbf{x})) = 0$ 。（注意我们是对 w_j 而非 w_i 求偏导。）在什么条件下这些非对角线元素为零呢？恰好当 f_i 和 g_i 关于 w_j 为常数时， $\frac{\partial}{\partial w_j} f_i(\mathbf{w}) = \frac{\partial}{\partial w_j} g_i(\mathbf{x}) = 0$ 。无论算子是什么，只要这些偏导数为零，运算结果就为零， $0 \circ 0 = 0$ 无论如何都成立，而且常数的偏导数为零。

当 f_i 和 g_i 不是 w_j 的函数时，这些偏导数即为零。我们知道逐元素运算意味着 f_i 纯粹是 w_i 的函数，而 g_i 纯粹是 x_i 的函数。例如， $\mathbf{w} + \mathbf{x}$ 求的是 $w_i + x_i$ 的和。因此， $f_i(\mathbf{w}) \circ g_i(\mathbf{x})$ 化简为 $f_i(w_i) \circ g_i(x_i)$ ，目标就变为 $\frac{\partial}{\partial w_j} f_i(w_i) = \frac{\partial}{\partial w_j} g_i(x_i) = 0$ 。当 $j \neq i$ 时， $f_i(w_i)$ 和 $g_i(x_i)$ 对于关于 w_j 求偏导的算子来说就像常数，所以非对角线上的偏导数均为零。（严格来说，记号 $f_i(w_i)$ 是对我们记号的一种滥用，因为 f_i 和 g_i 是向量的函数，而非单个元素的函数。我们本当写成类似 $\hat{f}_i(w_i) = f_i(\mathbf{w})$ 的形式，但那会让方程更加混乱，而且程序员习惯于函数重载，所以我们还是继续使用这种记号。）

我们稍后会利用这一简化，并把“ $f_i(\mathbf{w})$ 和 $g_i(\mathbf{x})$ 分别最多只能访问 w_i 和 x_i ”这一约束称为逐元素对角条件。

在此条件下，雅可比矩阵对角线上的元素为 $\frac{\partial}{\partial w_i}(f_i(w_i) \circ g_i(x_i))$ ：

$$\frac{\partial \mathbf{y}}{\partial \mathbf{w}} = \begin{bmatrix} \frac{\partial}{\partial w_1}(f_1(w_1) \circ g_1(x_1)) & & & 0 \\ & \frac{\partial}{\partial w_2}(f_2(w_2) \circ g_2(x_2)) & & \\ & & \dots & \\ 0 & & & \frac{\partial}{\partial w_n}(f_n(w_n) \circ g_n(x_n)) \end{bmatrix}$$

（大写的“0”是简写记号，表示所有非对角线元素均为 0。）

更简洁地，我们可以写成：

$$\frac{\partial \mathbf{y}}{\partial \mathbf{w}} = \text{diag} \left(\frac{\partial}{\partial w_1} (f_1(w_1) \circ g_1(x_1)), \frac{\partial}{\partial w_2} (f_2(w_2) \circ g_2(x_2)), \dots, \frac{\partial}{\partial w_n} (f_n(w_n) \circ g_n(x_n)) \right)$$

以及

$$\frac{\partial \mathbf{y}}{\partial \mathbf{x}} = \text{diag} \left(\frac{\partial}{\partial x_1} (f_1(w_1) \circ g_1(x_1)), \frac{\partial}{\partial x_2} (f_2(w_2) \circ g_2(x_2)), \dots, \frac{\partial}{\partial x_n} (f_n(w_n) \circ g_n(x_n)) \right)$$

其中 $\text{diag}(\mathbf{x})$ 构造一个矩阵，其对角线元素取自向量 $\mathbf{x}$ 。

由于我们会做大量简单的向量运算，二元逐元素运算中的一般函数 $\mathbf{f}(\mathbf{w})$ 往往就是向量 $\mathbf{w}$ 本身。只要一般函数是向量，我们就知道 $f_i(\mathbf{w})$ 会化简为 $f_i(w_i) = w_i$ 。例如，向量加法 $\mathbf{w} + \mathbf{x}$ 符合我们的逐元素对角条件，因为 $\mathbf{f}(\mathbf{w}) + \mathbf{g}(\mathbf{x})$ 的标量方程 $y_i = f_i(\mathbf{w}) + g_i(\mathbf{x})$ 可以化简为 $y_i = f_i(w_i) + g_i(x_i) = w_i + x_i$ ，其偏导数为：

$$\frac{\partial}{\partial w_i} (f_i(w_i) + g_i(x_i)) = \frac{\partial}{\partial w_i} (w_i + x_i) = 1 + 0 = 1$$

$$\frac{\partial}{\partial x_i} (f_i(w_i) + g_i(x_i)) = \frac{\partial}{\partial x_i} (w_i + x_i) = 0 + 1 = 1$$

这便得到 $\frac{\partial(\mathbf{w}+\mathbf{x})}{\partial \mathbf{w}} = \frac{\partial(\mathbf{w}+\mathbf{x})}{\partial \mathbf{x}} = I$ ，即单位矩阵，因为对角线上的每个元素都是 1。 I 表示适当维数的方阵单位矩阵，除对角线（全部为 1）外处处为零。

鉴于这个特殊情况（ $f_i(\mathbf{w})$ 化简为 $f_i(w_i)$ ）的简洁性，你应该能够推导出常见向量逐元素二元运算的雅可比矩阵：

Op Partial with respect to $\mathbf{w}$

$$+ \quad \frac{\partial(\mathbf{w}+\mathbf{x})}{\partial \mathbf{w}} = \text{diag}(\dots \frac{\partial(w_i+x_i)}{\partial w_i} \dots) = \text{diag}(\vec{1}) = I$$

$$- \quad \frac{\partial(\mathbf{w}-\mathbf{x})}{\partial \mathbf{w}} = \text{diag}(\dots \frac{\partial(w_i-x_i)}{\partial w_i} \dots) = \text{diag}(\vec{1}) = I$$

$$\otimes \quad \frac{\partial(\mathbf{w} \otimes \mathbf{x})}{\partial \mathbf{w}} = \text{diag}(\dots \frac{\partial(w_i \times x_i)}{\partial w_i} \dots) = \text{diag}(\mathbf{x})$$

$$\oslash \quad \frac{\partial(\mathbf{w} \oslash \mathbf{x})}{\partial \mathbf{w}} = \text{diag}(\dots \frac{\partial(w_i/x_i)}{\partial w_i} \dots) = \text{diag}(\dots \frac{1}{x_i} \dots)$$

Op Partial with respect to $\mathbf{x}$

$$+ \quad \frac{\partial(\mathbf{w}+\mathbf{x})}{\partial \mathbf{x}} = I$$

$$- \quad \frac{\partial(\mathbf{w}-\mathbf{x})}{\partial \mathbf{x}} = \text{diag}(\dots \frac{\partial(w_i-x_i)}{\partial x_i} \dots) = \text{diag}(-\vec{1}) = -I$$

$$\otimes \quad \frac{\partial(\mathbf{w} \otimes \mathbf{x})}{\partial \mathbf{x}} = \text{diag}(\mathbf{w})$$

$$\oslash \quad \frac{\partial(\mathbf{w} \oslash \mathbf{x})}{\partial \mathbf{x}} = \text{diag}(\dots \frac{-w_i}{x_i^2} \dots)$$

$\otimes$ 和 $\oslash$ 算子分别是逐元素乘法和除法； $\otimes$ 有时被称为哈达玛积。逐元素乘法和除法没有标准的记号，因此我们采用与一般二元运算记号一致的做法。

4.3 涉及标量扩展的导数

当我们把标量与向量相乘或相加时，实际上是在隐式地把标量扩展为向量，然后执行逐元素二元运算。例如，把标量 z 加到向量 $\mathbf{x}$ 上，即 $\mathbf{y} = \mathbf{x} + z$ ，实际上就是 $\mathbf{y} = \mathbf{f}(\mathbf{x}) + \mathbf{g}(z)$ ，其中 $\mathbf{f}(\mathbf{x}) = \mathbf{x}$ ， $\mathbf{g}(z) = \vec{1}z$ 。（记号 $\vec{1}$ 表示适当长度的全 1 向量。） z 是任意不依赖于 $\mathbf{x}$ 的标量，这很有用，因为这样一来对任意 x_i 都有 $\frac{\partial z}{\partial x_i} = 0$ ，从而简化我们的偏导数计算。（在我们这里的讨论中，把变量 z 看作常数是可以的。）类似地，乘以标量 $\mathbf{y} = \mathbf{x}z$ ，实际上就是 $\mathbf{y} = \mathbf{f}(\mathbf{x}) \otimes \mathbf{g}(z) = \mathbf{x} \otimes \vec{1}z$ ，其中 $\otimes$ 是两个向量的逐元素乘法（哈达玛积）。

向量-标量加法和乘法关于向量 $\mathbf{x}$ 的偏导数使用我们的逐元素法则：

$$\frac{\partial \mathbf{y}}{\partial \mathbf{x}} = \text{diag} \left(\dots \frac{\partial}{\partial x_i} (f_i(x_i) \oslash g_i(z)) \dots \right)$$

这是因为函数 $\mathbf{f}(\mathbf{x}) = \mathbf{x}$ 和 $\mathbf{g}(z) = \vec{1}z$ 显然满足我们关于雅可比矩阵的逐元素对角条件（即 $f_i(\mathbf{x})$ 最多只涉及 x_i ，而 $g_i(z)$ 涉及 $\vec{1}z$ 向量的第 i^{th} 个值）。

使用标量偏导数的通常法则，我们得到向量-标量加法对应的雅可比矩阵的如下对角线元素：

$$\frac{\partial}{\partial x_i} (f_i(x_i) + g_i(z)) = \frac{\partial (x_i + z)}{\partial x_i} = \frac{\partial x_i}{\partial x_i} + \frac{\partial z}{\partial x_i} = 1 + 0 = 1$$

因此， $\frac{\partial}{\partial \mathbf{x}} (\mathbf{x} + z) = \text{diag}(\vec{1}) = I$ 。

然而，关于标量参数 z 求偏导得到的是一个垂直向量，而不是对角矩阵。该向量的元素为：

$$\frac{\partial}{\partial z} (f_i(x_i) + g_i(z)) = \frac{\partial (x_i + z)}{\partial z} = \frac{\partial x_i}{\partial z} + \frac{\partial z}{\partial z} = 0 + 1 = 1$$

因此， $\frac{\partial}{\partial z} (\mathbf{x} + z) = \vec{1}$ 。

向量-标量乘法对应的雅可比矩阵的对角线元素涉及标量导数的乘积法则：

$$\frac{\partial}{\partial x_i} (f_i(x_i) \otimes g_i(z)) = x_i \frac{\partial z}{\partial x_i} + z \frac{\partial x_i}{\partial x_i} = 0 + z = z$$

因此， $\frac{\partial}{\partial \mathbf{x}} (\mathbf{x}z) = \text{diag}(\vec{1}z) = Iz$ 。

关于标量参数 z 的偏导数是一个垂直向量，其元素为：

$$\frac{\partial}{\partial z} (f_i(x_i) \otimes g_i(z)) = x_i \frac{\partial z}{\partial z} + z \frac{\partial x_i}{\partial z} = x_i + 0 = x_i$$

这便得到 $\frac{\partial}{\partial z} (\mathbf{x}z) = \mathbf{x}$ 。

4.4 向量求和归约

对向量的元素求和是深度学习中的一个重要运算，例如网络损失函数，但我们也可以用它来简化向量点积以及其他把向量归约为标量的运算的导数计算。

设 $y = \text{sum}(\mathbf{f}(\mathbf{x})) = \sum_{i=1}^n f_i(\mathbf{x})$ 。注意我们在这里特意把参数保留为向量 $\mathbf{x}$ ，因为每个函数 f_i 都可能使用向量中的全部值，而不仅仅是 x_i 。求和是对函数的结果求和，而不是对参数求和。向量求和的梯度 ($1 \times n$ 雅可比矩阵) 为：

$$\begin{aligned}\frac{\partial y}{\partial \mathbf{x}} &= \left[\frac{\partial y}{\partial x_1}, \frac{\partial y}{\partial x_2}, \dots, \frac{\partial y}{\partial x_n} \right] \\ &= \left[\frac{\partial}{\partial x_1} \sum_i f_i(\mathbf{x}), \frac{\partial}{\partial x_2} \sum_i f_i(\mathbf{x}), \dots, \frac{\partial}{\partial x_n} \sum_i f_i(\mathbf{x}) \right] \\ &= \left[\sum_i \frac{\partial f_i(\mathbf{x})}{\partial x_1}, \sum_i \frac{\partial f_i(\mathbf{x})}{\partial x_2}, \dots, \sum_i \frac{\partial f_i(\mathbf{x})}{\partial x_n} \right] \quad (\text{move derivative inside } \sum)\end{aligned}$$

(梯度元素内部的求和可能比较棘手，所以务必保持你的记号前后一致。)

让我们看看简单情形 $y = \text{sum}(\mathbf{x})$ 的梯度。求和内部的函数就是 $f_i(\mathbf{x}) = x_i$ ，于是梯度为：

$$\nabla y = \left[\sum_i \frac{\partial f_i(\mathbf{x})}{\partial x_1}, \sum_i \frac{\partial f_i(\mathbf{x})}{\partial x_2}, \dots, \sum_i \frac{\partial f_i(\mathbf{x})}{\partial x_n} \right] = \left[\sum_i \frac{\partial x_i}{\partial x_1}, \sum_i \frac{\partial x_i}{\partial x_2}, \dots, \sum_i \frac{\partial x_i}{\partial x_n} \right]$$

因为当 $j \neq i$ 时 $\frac{\partial}{\partial x_j} x_i = 0$ ，所以可以化简为：

$$\nabla y = \left[\frac{\partial x_1}{\partial x_1}, \frac{\partial x_2}{\partial x_2}, \dots, \frac{\partial x_n}{\partial x_n} \right] = [1, 1, \dots, 1] = \vec{1}^T$$

注意，结果是一个全为 1 的水平向量，而不是垂直向量，因此梯度为 $\vec{1}^T$ 。（ $\vec{1}^T$ 中的 T 指数表示所指向量的转置。在这里，它把一个垂直向量翻转成水平向量。）保持所有向量和矩阵的形状一致非常重要，否则就不可能计算复杂函数的导数。

再举一个例子，让我们对向量乘以常数标量的结果求和。如果 $y = \text{sum}(\mathbf{x}z)$ ，那么 $f_i(\mathbf{x}, z) = x_i z$ 。梯度为：

$$\begin{aligned}\frac{\partial y}{\partial \mathbf{x}} &= \left[\sum_i \frac{\partial}{\partial x_1} x_i z, \sum_i \frac{\partial}{\partial x_2} x_i z, \dots, \sum_i \frac{\partial}{\partial x_n} x_i z \right] \\ &= \left[\frac{\partial}{\partial x_1} x_1 z, \frac{\partial}{\partial x_2} x_2 z, \dots, \frac{\partial}{\partial x_n} x_n z \right] \\ &= [z, z, \dots, z]\end{aligned}$$

关于标量变量 z 的导数是 1×1 的:

$$\begin{aligned}\frac{\partial y}{\partial z} &= \frac{\partial}{\partial z} \sum_{i=1}^n x_i z \\&= \sum_i \frac{\partial}{\partial z} x_i z \\&= \sum_i x_i \\&= \text{sum}(\mathbf{x})\end{aligned}$$

4.5 链式法则

我们无法仅凭目前所见的基本矩阵微积分规则来计算非常复杂函数的偏导数。例如,对于 $\text{sum}(\mathbf{w} + \mathbf{x})$ 这样的嵌套表达式,若不将其化简为等价的标量形式,就无法直接求导。我们需要利用我们称之为向量链式法则的规则来组合基本的向量规则。遗憾的是,有许多求导规则都归入“链式法则”名下,因此我们必须谨慎区分我们所说的是哪一种链式法则。我们这里的目标之一是清晰地定义并命名三种不同的链式法则,并指出它们各自适用的情形。为了热身,我们先从我们称之为单变量链式法则的内容开始,即我们希望求标量函数关于标量的导数。然后,我们将介绍一个重要的概念,即全导数,并利用它来定义我们严格称之为单变量全导数链式法则的法则。接着,我们就能以神经网络所需的完整形态来迎接向量链式法则。

链式法则在概念上是一种分而治之的策略(类似于快速排序),它把复杂的表达式拆分成导数更容易计算的子表达式。其威力源于这样一个事实:我们可以孤立地处理每个简单的子表达式,却仍能组合中间结果,得到正确的整体结果。

当我们需要对由嵌套子表达式组成的表达式求导时,链式法则就派上了用场。例如,面对 $\frac{d}{dx} \sin(x^2)$ 这样的表达式时,我们就需要链式法则。最外层的表达式对中间结果取 $\sin$,而这个中间结果是对 x 求平方的嵌套子表达式。具体来说,我们需要的是单变量链式法则,那么让我们先来深入探讨它。

4.5.1 单变量链式法则

我们先从嵌套表达式的导数解法入手: $\frac{d}{dx} \sin(x^2) = 2x \cos(x^2)$ 。不需要数学天才也能看出,解法的组成部分带有标量求导规则的痕迹: $\frac{d}{dx} x^2 = 2x$ 和 $\frac{d}{du} \sin(u) = \cos(u)$ 。看起来解法就是把外层表达式的导数乘以内层表达式的导数,即“把各部分链接起来”,事实正是如此。在本节中,我们将探究其中起作用的一般原理,并提供一种适用于高度嵌套的单变量表达式的求解流程。

链式法则通常以嵌套函数的形式定义，例如单变量链式法则中的 $y = f(g(x))$ 。（你也会看到用函数复合 $(f \circ g)(x)$ 来定义链式法则，二者是一回事。）有些资料用简写记号 $y' = f'(g(x))g'(x)$ 来书写导数，但这掩盖了我们正在引入一个中间变量的事实： $u = g(x)$ ，这一点我们很快就会看到。最好显式地定义 $f(g(x))$ 的**单变量链式法则**，这样我们就永远不会对错误的变量求导。下面是我们推荐的单变量链式法则的表述：

$$\frac{dy}{dx} = \frac{dy}{du} \frac{du}{dx}$$

要运用单变量链式法则，请遵循以下步骤：

1. 为嵌套子表达式以及二元和一元算子的子表达式引入中间变量；例如， $\times$ 是二元算子， $\sin(x)$ 和其他三角函数通常是一元算子，因为只有一个操作数。这一步将所有方程规范化为单个算子或函数应用。
2. 计算中间变量关于其参数的导数。
3. 将所有中间变量的导数相乘合并，得到整体结果。
4. 如果导数方程中引用了中间变量，则将其代回。

第三步正是“链式法则”中“链”的由来，因为它把中间结果链接在一起。将所有中间导数相乘是所有链式法则变体的共同主题。

让我们在 $y = f(g(x)) = \sin(x^2)$ 上尝试这一流程：

1. 引入中间变量。令 $u = x^2$ 表示子表达式 x^2 （即 $u(x) = x^2$ 的简写）。于是我们得到：

$$\begin{aligned} u &= x^2 && \text{(relative to definition } f(g(x)), g(x) = x^2) \\ y &= \sin(u) && (y = f(u) = \sin(u)) \end{aligned}$$

这些子表达式的顺序不影响答案，但我们建议按照嵌套所决定的运算顺序的逆序（由内到外）进行。这样，表达式和导数始终是先前已计算元素的函数。

2. 计算导数。

$$\begin{aligned} \frac{du}{dx} &= 2x && \text{(Take derivative with respect to } x) \\ \frac{dy}{du} &= \cos(u) && \text{(Take derivative with respect to } u \text{ not } x) \end{aligned}$$

3. 合并。

$$\frac{dy}{dx} = \frac{dy}{du} \frac{du}{dx} = \cos(u)2x$$

4. 代回。

$$\frac{dy}{dx} = \frac{dy}{du} \frac{du}{dx} = \cos(x^2)2x = 2x\cos(x^2)$$

注意到孤立地计算中间变量的导数有多么容易！链式法则表明这样做是合法的，并告诉我们如何组合中间结果，从而得到 $2x\cos(x^2)$ 。

你可以从单位相消的角度来理解链式法则的合并步骤。如果我们令 y 为英里， x 为油箱中的加仑数， u 为加仑，那么我们可以把 $\frac{dy}{dx} = \frac{dy}{du} \frac{du}{dx}$ 解释为 $\frac{\text{miles}}{\text{tank}} = \frac{\text{miles}}{\text{gallon}} \frac{\text{gallon}}{\text{tank}}$ 。gallon 的分母与分子相消。

思考单变量链式法则的另一种方式是，把整体表达式可视化为数据流图或运算链（对编译器领域的人而言，则是[抽象语法树](#)）：

$$\begin{array}{c} y = \sin \\ \uparrow \\ u = \text{square} \\ \uparrow \\ x \end{array} \left\{ \begin{array}{l} \frac{dy}{du} \\ \frac{du}{dx} \end{array} \right.$$

函数参数 x 的变化会经过平方运算向上传播，再经过 $\sin$ 运算，最终改变结果 y 。你可以把 $\frac{du}{dx}$ 理解为“获取从 x 到 u 的变化”，把 $\frac{dy}{du}$ 理解为“获取从 u 到 y 的变化”。从 x 到 y 需要经过一次中间跳转。按照惯例，链式法则通常写成从输出变量向下到参数的形式： $\frac{dy}{dx} = \frac{dy}{du} \frac{du}{dx}$ 。但是，如果我们反转流动方向并使用等价的 $\frac{dy}{dx} = \frac{du}{dx} \frac{dy}{du}$ ，从 x 到 y 的视角会更加清晰。

单变量链式法则适用的条件。注意，从 x 到根节点 y 只有一条数据流路径。 x 的变化只能以一种方式影响输出 y 。这就是我们可以应用单变量链式法则的条件。还有一个更容易记住（虽然略为宽松）的条件：中间子表达式函数 $u(x)$ 和 $y(u)$ 都没有多于一个参数。考虑 $y(x) = x + x^2$ ，引入中间变量 u 后，它会变成 $y(x, u) = x + u$ 。正如我们将在下一节看到的， $y(x, u)$ 从 x 到 y 有多条路径。为了处理这种情况，我们将运用单变量全导数链式法则。

顺便为那些对自动微分感兴趣的读者说明一下：论文和库文档中使用的术语是前向微分和反向微分（用于反向传播算法）。从数据流的角度看，我们计算的是前向微分，因为它遵循正常的数据流方向。反向微分自然沿另一个方向进行，我们问的是输出的变化会如何影响函数参数 x 。由于反向微分可以一次性确定所有函数参数的变化，因此计算含大量参数的函数的导数时，它要高效得多。而前向微分则必须依次考虑每个参数的变化如何影响函数输出 y 。下表强调了这两种技术在偏导数计算顺序上的差异。

从 x 到 y 的前向微分 从 y 到 x 的反向微分

$$\frac{dy}{dx} = \frac{du}{dx} \frac{dy}{du}$$

$$\frac{dy}{dx} = \frac{dy}{du} \frac{du}{dx}$$

自动微分超出了本文的范围，但我们正在为将来的文章做铺垫。

许多读者可以心算出 $\frac{d}{dx} \sin(x^2)$ ，但我们的目标是找到一种即使对非常复杂的表达式也行之有效的流程。这一流程也正是 PyTorch 等库中[自动微分](#)的工作原理。因此，通过以这种方式手动求解导数，你同时也在学习如何在 PyTorch 中为自定义神经网络定义函数。

对于深度嵌套的表达式，不妨把运用链式法则想成编译器将 $f_4(f_3(f_2(f_1(x))))$ 之类的嵌套函数调用展开为一系列（链式）调用。调用函数 f_i 的结果被保存到一个称为寄存器的临时变量中，然后作为参数传给 f_{i+1} 。让我们用我们的流程处理一个高度嵌套的方程 $y = f(x) = \ln(\sin(x^3)^2)$ ，看看实际效果：

1. 引入中间变量。

$$\begin{aligned} u_1 &= f_1(x) = x^3 \\ u_2 &= f_2(u_1) = \sin(u_1) \\ u_3 &= f_3(u_2) = u_2^2 \\ u_4 &= f_4(u_3) = \ln(u_3) (y = u_4) \end{aligned}$$

2. 计算导数。

$$\begin{aligned} \frac{d}{u_1} u_1 &= \frac{d}{x} x^3 = 3x^2 \\ \frac{d}{u_1} u_2 &= \frac{d}{u_1} \sin(u_1) = \cos(u_1) \\ \frac{d}{u_2} u_3 &= \frac{d}{u_2} u_2^2 = 2u_2 \\ \frac{d}{u_3} u_4 &= \frac{d}{u_3} \ln(u_3) = \frac{1}{u_3} \end{aligned}$$

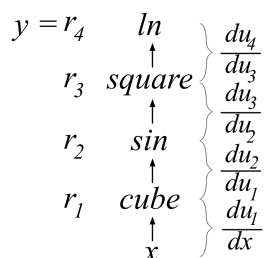
3. 合并四个中间值。

$$\frac{dy}{dx} = \frac{du_4}{dx} = \frac{du_4}{du_3} \frac{du_3}{du_2} \frac{du_2}{du_1} \frac{du_1}{dx} = \frac{1}{u_3} 2u_2 \cos(u_1) 3x^2 = \frac{6u_2 x^2 \cos(u_1)}{u_3}$$

4. 代回。

$$\frac{dy}{dx} = \frac{6 \sin(u_1) x^2 \cos(x^3)}{u_2^2} = \frac{6 \sin(x^3) x^2 \cos(x^3)}{\sin(u_1)^2} = \frac{6 \sin(x^3) x^2 \cos(x^3)}{\sin(x^3)^2} = \frac{6 x^2 \cos(x^3)}{\sin(x^3)}$$

下面是从 x 到 y 经过运算链的数据流可视化：



到目前为止，我们可以使用链式法则处理单变量 x 的嵌套表达式的导数，但前提是 x 只能通过单一数据流路径影响 y 。为了处理更复杂的表达式，我们需要扩展我们的方法，这将在下一步进行。

4.5.2 单变量全导数链式法则

我们的单变量链式法则适用性有限，因为所有中间变量都必须是单变量的函数。但它展示了链式法则的核心机制：将所有中间子表达式的导数相乘。然而，为了处理 $y = f(x) = x + x^2$ 这样更一般的表达式，我们需要对这条基本链式法则进行扩充。

当然，我们立刻就能看出 $\frac{dy}{dx} = \frac{d}{dx}x + \frac{d}{dx}x^2 = 1 + 2x$ ，但这用的是标量加法求导法则，而不是链式法则。如果我们试图套用单变量链式法则，就会得到错误的答案。事实上，之前的链式法则在这种情况下毫无意义，因为求导算子 $\frac{d}{dx}$ 不适用于多元函数，例如我们中间变量中的 u_2 ：

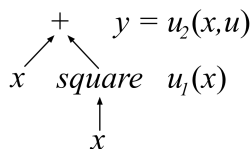
$$\begin{aligned} u_1(x) &= x^2 \\ u_2(x, u_1) &= x + u_1 \quad (y = f(x) = u_2(x, u_1)) \end{aligned}$$

我们还是试着套用一下，看看会发生什么。如果我们假装 $\frac{du_2}{du_1} = 0 + 1 = 1$ 且 $\frac{du_1}{dx} = 2x$ ，那么 $\frac{dy}{dx} = \frac{du_2}{dx} = \frac{du_2}{du_1} \frac{du_1}{dx} = 2x$ ，而不是正确答案 $1 + 2x$ 。

由于 $u_2(x, u) = x + u_1$ 有多个参数，偏导数开始发挥作用。让我们盲目地把偏导数算子应用到我们所有的方程上，看看会得到什么：

$$\begin{aligned} \frac{\partial u_1(x)}{\partial x} &= 2x && \text{(same as } \frac{du_1(x)}{dx} \text{)} \\ \frac{\partial u_2(x, u_1)}{\partial u_1} &= \frac{\partial}{\partial u_1}(x + u_1) = 0 + 1 = 1 \\ \frac{\partial u_2(x, u_1)}{\partial x} &\text{⊗} \frac{\partial}{\partial x}(x + u_1) = 1 + 0 = 1 && \text{(something's not quite right here!)} \end{aligned}$$

哎呀！偏导数 $\frac{\partial u_2(x, u_1)}{\partial x}$ 是错误的，因为它违反了偏导数的一个关键假设。当对 x 求偏导时，其他变量不能随 x 的变化而变化。否则，我们就无法把其他变量当作常数来处理。但显然， $u_1(x) = x^2$ 是 x 的函数，因此会随 x 变化。 $\frac{\partial u_2(x, u_1)}{\partial x} \neq 1 + 0$ ，因为 $\frac{\partial u_1(x)}{\partial x} \neq 0$ 。快速看一眼 $y = u_2(x, u_1)$ 的数据流图，就会发现从 x 到 y 有多条路径，因此很明显，我们需要考虑对 x 的直接依赖和间接依赖（通过 $u_1(x)$ ）：



x 的变化既通过作为加法操作数影响 y ，也通过作为平方算子的操作数影响 y 。下面这个方程描述

了 x 的微调如何影响输出:

$$\hat{y} = (x + \Delta x) + (x + \Delta x)^2$$

那么, $\Delta y = \hat{y} - y$, 我们可以把它读作“ y 的变化量等于原始 y 与微调 x 后的 y 之差”。

如果我们令 $x = 1$, 则 $y = 1 + 1^2 = 2$ 。如果把 x 增加 1, 即 $\Delta x = 1$, 则 $\hat{y} = (1+1) + (1+1)^2 = 2 + 4 = 6$ 。
 y 的变化量并不是 $\partial u_2 / u_1$ 会让我们相信的 1, 而是 $6 - 2 = 4$!

接下来是[全导数](#)的“法则”, 它大致是说: 要计算 $\frac{dy}{dx}$, 我们需要把 x 的变化对 y 的变化的所有可能贡献求和。关于 x 的全导数假定所有变量 (例如本例中的 u_1) 都是 x 的函数, 并可能随 x 的变化而变化。通过中间变量 $u_1(x)$ 直接和间接依赖于 x 的 $f(x) = u_2(x, u_1)$ 的全导数为:

$$\frac{dy}{dx} = \frac{\partial f(x)}{\partial x} = \frac{\partial u_2(x, u_1)}{\partial x} = \frac{\partial u_2}{\partial x} \frac{\partial x}{\partial x} + \frac{\partial u_2}{\partial u_1} \frac{\partial u_1}{\partial x} = \frac{\partial u_2}{\partial x} + \frac{\partial u_2}{\partial u_1} \frac{\partial u_1}{\partial x}$$

使用这个公式, 我们得到正确的答案:

$$\frac{dy}{dx} = \frac{\partial f(x)}{\partial x} = \frac{\partial u_2}{\partial x} + \frac{\partial u_2}{\partial u_1} \frac{\partial u_1}{\partial x} = 1 + 1 \times 2x = 1 + 2x$$

这就是我们称之为单变量全导数链式法则的一个应用:

$$\frac{\partial f(x, u_1, \dots, u_n)}{\partial x} = \frac{\partial f}{\partial x} + \frac{\partial f}{\partial u_1} \frac{\partial u_1}{\partial x} + \frac{\partial f}{\partial u_2} \frac{\partial u_2}{\partial x} + \dots + \frac{\partial f}{\partial u_n} \frac{\partial u_n}{\partial x} = \frac{\partial f}{\partial x} + \sum_{i=1}^n \frac{\partial f}{\partial u_i} \frac{\partial u_i}{\partial x}$$

全导数假定所有变量都可能相互依赖, 而偏导数则假定除 x 之外的所有变量都是常数。

这里的记号有些微妙之处。所有导数都写成偏导数的形式, 因为 f 和 u_i 是多元变量的函数。这种记号与 [MathWorld](#) 的记号一致, 但与[维基百科](#)不同, 后者使用的是 $df(x, u_1, \dots, u_n)/dx$ (可能是为了强调方程的全导数性质)。我们将坚持使用偏导数记号, 以便与下一节中关于向量链式法则的讨论保持一致。

在实践中, 只需记住: 当你对 x 取全导数时, 其他变量可能也是 x 的函数, 因此也要把它们的贡献加进来。方程左边看起来像典型的偏导数, 但右边实际上就是全导数。不过, 许多临时变量通常是单参数的函数, 这意味着单变量全导数链式法则会退化为单变量链式法则。

让我们看一个嵌套子表达式, 例如 $f(x) = \sin(x + x^2)$ 。我们引入三个中间变量:

$$\begin{aligned} u_1(x) &= x^2 \\ u_2(x, u_1) &= x + u_1 \\ u_3(u_2) &= \sin(u_2) \quad (y = f(x) = u_3(u_2)) \end{aligned}$$

以及偏导数:

$$\begin{aligned} \frac{\partial u_1}{\partial x} &= 2x \\ \frac{\partial u_2}{\partial x} &= \frac{\partial x}{\partial x} + \frac{\partial u_2}{\partial u_1} \frac{\partial u_1}{\partial x} = 1 + 1 \times 2x = 1 + 2x \\ \frac{\partial f(x)}{\partial x} &= \frac{\partial u_3}{\partial x} + \frac{\partial u_3}{\partial u_2} \frac{\partial u_2}{\partial x} = 0 + \cos(u_2) \frac{\partial u_2}{\partial x} = \cos(x + x^2)(1 + 2x) \end{aligned}$$

其中 $\frac{\partial u_2}{\partial x}$ 和 $\frac{\partial f(x)}{\partial x}$ 都含有考虑全导数的 $\frac{\partial u_i}{\partial x}$ 项。

还要注意，全导数公式始终是对 $\frac{\partial f}{\partial u_i} \frac{\partial u_i}{\partial x}$ 各项**求和**，而不是相乘。人们很容易认为对导数中的各项求和是合理的，因为例如 $y = x + x^2$ 就是两项相加。并非如此。全导数之所以对各项求和，是因为它表示 x 对 y 的变化的所有贡献的加权和。例如，给定 $y = x \times x^2$ 而不是 $y = x + x^2$ ，全导数链式法则公式仍然是对偏导数项求和。（ $x \times x^2$ 可以化简为 x^3 ，但为了演示，我们暂不合并这些项。）下面是中间变量和偏导数：

$$\begin{aligned} u_1(x) &= x^2 \\ u_2(x, u_1) &= xu_1 \quad (y = f(x) = u_2(x, u_1)) \\ \frac{\partial u_1}{\partial x} &= 2x \\ \frac{\partial u_2}{\partial x} &= u_1 \quad (\text{for } u_2 = x + u_1, \frac{\partial u_2}{\partial x} = 1) \\ \frac{\partial u_2}{\partial u_1} &= x \quad (\text{for } u_2 = x + u_1, \frac{\partial u_2}{\partial u_1} = 1) \end{aligned}$$

然而，全导数的形式保持不变：

$$\frac{dy}{dx} = \frac{\partial u_2}{\partial x} + \frac{\partial u_2}{\partial u_1} \frac{du_1}{dx} = u_1 + x2x = x^2 + 2x^2 = 3x^2$$

当中间变量的算子改变时，改变的是偏导数（权重），而不是公式。

微积分功底扎实的读者可能会疑惑：为什么我们即使对 $x + x^2$ 中的 x^2 这样的非嵌套子表达式，也要积极地引入中间变量。我们采用这一流程有三个原因：(i) 对化简后的子表达式求导通常很简单；(ii) 可以简化链式法则；(iii) 这一流程与神经网络库中自动微分的工作方式相吻合。

让我们更积极地使用中间变量，看看如何把单变量全导数链式法则化简为最终形式。目标是去掉突兀地伸在前面的 $\frac{\partial f}{\partial x}$ ：

$$\frac{\partial f(x, u_1, \dots, u_n)}{\partial x} = \frac{\partial f}{\partial x} + \sum_{i=1}^n \frac{\partial f}{\partial u_i} \frac{\partial u_i}{\partial x}$$

实现这一点的简单办法是引入一个新的临时变量作为 x 的别名： $u_{n+1} = x$ 。于是，公式就化简为我们最终的形式：

$$\frac{\partial f(u_1, \dots, u_{n+1})}{\partial x} = \sum_{i=1}^{n+1} \frac{\partial f}{\partial u_i} \frac{\partial u_i}{\partial x}$$

这条考虑全导数的链式法则，在所有中间变量都是单变量函数时会退化为单变量链式法则。因此，你可以记住这个更通用的公式来涵盖两种情况。稍微剧透一下：注意这个求和式看起来很像向量点积 $\frac{\partial f}{\partial \mathbf{u}} \cdot \frac{\partial \mathbf{u}}{\partial x}$ ，或者向量乘法 $\frac{\partial f}{\partial \mathbf{u}} \frac{\partial \mathbf{u}}{\partial x}$ 。

在继续之前，关于网络上的术语要提醒一句。遗憾的是，本节给出的基于全导数的链式法则，在微积分讨论中被普遍称为“多元链式法则”，这极具误导性！只有中间变量才是多元函数。整体函数，例

如 $f(x) = x + x^2$ ，是一个只接受单个参数 x 的标量函数。导数和参数都是标量而非向量，这与所谓的多元链式法则给人的预期不同。（在非矩阵微积分课程的情境下，“多元链式法则”很可能没有歧义。）为了减少混淆，我们使用“单变量全导数链式法则”来阐明简单的单变量链式法则 $\frac{dy}{dx} = \frac{dy}{du} \frac{du}{dx}$ 与本节这条法则之间的区别特征。

4.5.3 向量链式法则

既然我们已经很好地掌握了全导数链式法则，现在可以着手处理函数向量和向量变量的链式法则了。令人惊讶的是，这条更一般的链式法则看起来和标量的单变量链式法则一样简单。与其直接给出向量链式法则，不如让我们自己重新发现它，从而牢牢掌握它。我们可以先计算示例向量函数 $\mathbf{y} = \mathbf{f}(x)$ 关于标量的导数，看看能否抽象出一个通用公式。

$$\begin{bmatrix} y_1(x) \\ y_2(x) \end{bmatrix} = \begin{bmatrix} f_1(x) \\ f_2(x) \end{bmatrix} = \begin{bmatrix} \ln(x^2) \\ \sin(3x) \end{bmatrix}$$

让我们引入两个中间变量 g_1 和 g_2 ，每个 f_i 对应一个，这样 y 看起来更像 $\mathbf{y} = \mathbf{f}(\mathbf{g}(x))$ ：

$$\begin{bmatrix} g_1(x) \\ g_2(x) \end{bmatrix} = \begin{bmatrix} x^2 \\ 3x \end{bmatrix}$$

$$\begin{bmatrix} f_1(\mathbf{g}) \\ f_2(\mathbf{g}) \end{bmatrix} = \begin{bmatrix} \ln(g_1) \\ \sin(g_2) \end{bmatrix}$$

向量 $\mathbf{y}$ 关于标量 x 的导数是一个列向量，其元素使用单变量全导数链式法则计算：

$$\frac{\partial \mathbf{y}}{\partial x} = \begin{bmatrix} \frac{\partial f_1(\mathbf{g})}{\partial x} \\ \frac{\partial f_2(\mathbf{g})}{\partial x} \end{bmatrix} = \begin{bmatrix} \frac{\partial f_1}{\partial g_1} \frac{\partial g_1}{\partial x} + \frac{\partial f_1}{\partial g_2} \frac{\partial g_2}{\partial x} \\ \frac{\partial f_2}{\partial g_1} \frac{\partial g_1}{\partial x} + \frac{\partial f_2}{\partial g_2} \frac{\partial g_2}{\partial x} \end{bmatrix} = \begin{bmatrix} \frac{1}{g_1} 2x + 0 \\ 0 + \cos(g_2) 3 \end{bmatrix} = \begin{bmatrix} \frac{2x}{x^2} \\ 3\cos(3x) \end{bmatrix} = \begin{bmatrix} \frac{2}{x} \\ 3\cos(3x) \end{bmatrix}$$

好了，现在我们只用标量规则就得到了答案，只不过导数被分组到了一个向量中。让我们尝试从该结果中抽象出它的向量形式。目标是把下面的标量运算向量转换为向量运算。

$$\begin{bmatrix} \frac{\partial f_1}{\partial g_1} \frac{\partial g_1}{\partial x} + \frac{\partial f_1}{\partial g_2} \frac{\partial g_2}{\partial x} \\ \frac{\partial f_2}{\partial g_1} \frac{\partial g_1}{\partial x} + \frac{\partial f_2}{\partial g_2} \frac{\partial g_2}{\partial x} \end{bmatrix}$$

如果我们将 $\frac{\partial f_i}{\partial g_j} \frac{\partial g_j}{\partial x}$ 各项拆分，把 $\frac{\partial g_j}{\partial x}$ 项单独提取到一个向量中，就得到矩阵乘以向量：

$$\begin{bmatrix} \frac{\partial f_1}{\partial g_1} & \frac{\partial f_1}{\partial g_2} \\ \frac{\partial f_2}{\partial g_1} & \frac{\partial f_2}{\partial g_2} \end{bmatrix} \begin{bmatrix} \frac{\partial g_1}{\partial x} \\ \frac{\partial g_2}{\partial x} \end{bmatrix} = \frac{\partial \mathbf{f}}{\partial \mathbf{g}} \frac{\partial \mathbf{g}}{\partial x}$$

这意味着雅可比矩阵是两个其他雅可比矩阵的乘积，这还挺酷的。让我们验证一下结果：

$$\frac{\partial \mathbf{f}}{\partial \mathbf{g}} \frac{\partial \mathbf{g}}{\partial x} = \begin{bmatrix} \frac{1}{g_1} & 0 \\ 0 & \cos(g_2) \end{bmatrix} \begin{bmatrix} 2x \\ 3 \end{bmatrix} = \begin{bmatrix} \frac{1}{g_1} 2x + 0 \\ 0 + \cos(g_2) 3 \end{bmatrix} = \begin{bmatrix} \frac{2}{x} \\ 3\cos(3x) \end{bmatrix}$$

呼！我们得到了与标量方法相同的答案。这个针对函数向量和单一参数的向量链式法则看起来是正确的，而且确实与单变量链式法则相呼应。比较一下向量法则：

$$\frac{\partial}{\partial x} \mathbf{f}(\mathbf{g}(x)) = \frac{\partial \mathbf{f}}{\partial \mathbf{g}} \frac{\partial \mathbf{g}}{\partial x}$$

与单变量链式法则：

$$\frac{d}{dx} f(g(x)) = \frac{df}{dg} \frac{dg}{dx}$$

要使这个公式适用于多个参数或向量 $\mathbf{x}$ ，我们只需把方程中的 x 换成向量 $\mathbf{x}$ 。其效果是： $\frac{\partial \mathbf{g}}{\partial \mathbf{x}}$ 以及由此得到的雅可比矩阵 $\frac{\partial \mathbf{f}}{\partial \mathbf{x}}$ 现在是矩阵而不是列向量。我们完整的向量链式法则是：

$$\frac{\partial}{\partial \mathbf{x}} \mathbf{f}(\mathbf{g}(\mathbf{x})) = \frac{\partial \mathbf{f}}{\partial \mathbf{g}} \frac{\partial \mathbf{g}}{\partial \mathbf{x}} \quad (\text{Note: matrix multiply doesn't commute; order of } \frac{\partial \mathbf{f}}{\partial \mathbf{g}} \frac{\partial \mathbf{g}}{\partial \mathbf{x}} \text{ matters})$$

向量公式相对于单变量链式法则的妙处在于，它自动考虑了全导数，同时保持了相同的记号简洁性。雅可比矩阵包含了 f_i 关于 g_j 以及 g_i 关于 x_j 的所有可能组合。为完整起见，以下给出两个雅可比分量的完整形式：

$$\frac{\partial}{\partial \mathbf{x}} \mathbf{f}(\mathbf{g}(\mathbf{x})) = \begin{bmatrix} \frac{\partial f_1}{\partial g_1} & \frac{\partial f_1}{\partial g_2} & \cdots & \frac{\partial f_1}{\partial g_k} \\ \frac{\partial f_2}{\partial g_1} & \frac{\partial f_2}{\partial g_2} & \cdots & \frac{\partial f_2}{\partial g_k} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial g_1} & \frac{\partial f_m}{\partial g_2} & \cdots & \frac{\partial f_m}{\partial g_k} \end{bmatrix} \begin{bmatrix} \frac{\partial g_1}{\partial x_1} & \frac{\partial g_1}{\partial x_2} & \cdots & \frac{\partial g_1}{\partial x_n} \\ \frac{\partial g_2}{\partial x_1} & \frac{\partial g_2}{\partial x_2} & \cdots & \frac{\partial g_2}{\partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial g_k}{\partial x_1} & \frac{\partial g_k}{\partial x_2} & \cdots & \frac{\partial g_k}{\partial x_n} \end{bmatrix}$$

where $m = |\mathbf{f}|$, $n = |\mathbf{x}|$, and $k = |\mathbf{g}|$. The resulting Jacobian is $m \times n$ (an $m \times k$ matrix multiplied by a $k \times n$ matrix).

即使在这个 $\frac{\partial \mathbf{f}}{\partial \mathbf{g}} \frac{\partial \mathbf{g}}{\partial \mathbf{x}}$ 公式中，我们还可以进一步简化，因为在许多应用中，雅可比矩阵是方阵 ($m = n$) 且非对角元素为零。神经网络的性质决定了其相关数学处理的是向量的函数，而非函数的向量。例如，神经元仿射函数包含项 $\text{sum}(\mathbf{w} \otimes \mathbf{x})$ ，激活函数是 $\text{max}(0, \mathbf{x})$ ；我们将在下一节考虑这些函数的导数。

正如我们在前面一节所见，对向量 $\mathbf{w}$ 和 $\mathbf{x}$ 的逐元素运算产生对角线元素为 $\frac{\partial w_i}{\partial x_i}$ 的对角矩阵，因为 w_i 纯粹是 x_i 的函数，而非 x_j ($j \neq i$) 的函数。当 f_i 纯粹是 g_i 的函数且 g_i 纯粹是 x_i 的函数时，这里也会发生同样的情况：

$$\frac{\partial \mathbf{f}}{\partial \mathbf{g}} = \text{diag}\left(\frac{\partial f_i}{\partial g_i}\right)$$

$$\frac{\partial \mathbf{g}}{\partial \mathbf{x}} = \text{diag}\left(\frac{\partial g_i}{\partial x_i}\right)$$

In this situation, the vector chain rule simplifies to:

$$\frac{\partial}{\partial \mathbf{x}} \mathbf{f}(\mathbf{g}(\mathbf{x})) = \text{diag}\left(\frac{\partial f_i}{\partial g_i}\right) \text{diag}\left(\frac{\partial g_i}{\partial x_i}\right) = \text{diag}\left(\frac{\partial f_i}{\partial g_i} \frac{\partial g_i}{\partial x_i}\right)$$

Therefore, the Jacobian reduces to a diagonal matrix whose elements are the single-variable chain rule values.

在啃完所有这些数学之后，回报来了。你所需要的只是向量链式法则，因为单变量公式是向量链式法则的特例。下表总结了为得到雅可比矩阵所需相乘的各个分量。

$\frac{\partial}{\partial \mathbf{x}} \mathbf{f}(\mathbf{g}(\mathbf{x})) = \frac{\partial \mathbf{f}}{\partial \mathbf{g}} \frac{\partial \mathbf{g}}{\partial \mathbf{x}}$		scalar x		vector $\mathbf{x}$	
		scalar u	vector $\mathbf{u}$	vector $\mathbf{u}$	
scalar f	$\frac{\partial f}{\partial u}$	$\frac{\partial u}{\partial x}$	$\frac{\partial f}{\partial \mathbf{u}}$	$\frac{\partial \mathbf{u}}{\partial x}$	
vector $\mathbf{f}$	$\frac{\partial \mathbf{f}}{\partial u}$	$\frac{\partial \mathbf{u}}{\partial x}$	$\frac{\partial \mathbf{f}}{\partial \mathbf{u}}$	$\frac{\partial \mathbf{u}}{\partial x}$	

5 神经元激活的梯度

现在，我们已拥有计算单个神经网络计算单元的典型神经元激活相对于模型参数 $\mathbf{w}$ 和 b 的导数所需的全部要素：

$$activation(\mathbf{x}) = \max(0, \mathbf{w} \cdot \mathbf{x} + b)$$

(这代表一个具有全连接权重和修正线性单元 (ReLU) 激活的神经元。不过，还有其他仿射函数（如卷积）以及其他激活函数（如指数线性单元），它们遵循类似的逻辑。）

我们稍后再考虑 $\max$ ，先专注于计算 $\frac{\partial}{\partial \mathbf{w}}(\mathbf{w} \cdot \mathbf{x} + b)$ 和 $\frac{\partial}{\partial b}(\mathbf{w} \cdot \mathbf{x} + b)$ 。（回想一下，神经网络是通过优化其权重和偏置来学习的。）我们还没有讨论点积的导数 $y = \mathbf{f}(\mathbf{w}) \cdot \mathbf{g}(\mathbf{x})$ ，但我们可以利用链式法则，从而不必再记忆一条新规则。（注意记号是 y 而非 $\mathbf{y}$ ，因为结果是标量而不是向量。）

点积 $\mathbf{w} \cdot \mathbf{x}$ 就是对各元素逐元素相乘的结果求和： $\sum_i^n (w_i x_i) = \text{sum}(\mathbf{w} \otimes \mathbf{x})$ 。（你或许也会觉得记住线性代数记号 $\mathbf{w} \cdot \mathbf{x} = \mathbf{w}^T \mathbf{x}$ 很有用。）我们知道如何计算 $\text{sum}(\mathbf{x})$ 和 $\mathbf{w} \otimes \mathbf{x}$ 的偏导数，但还没有考

察 $sum(\mathbf{w} \otimes \mathbf{x})$ 的偏导数。为此我们需要链式法则，于是我们可以像使用单变量链式法则时那样引入一个中间向量变量 $\mathbf{u}$ ：

$$\begin{aligned}\mathbf{u} &= \mathbf{w} \otimes \mathbf{x} \\ y &= sum(\mathbf{u})\end{aligned}$$

一旦我们改写 y ，就能识别出两个我们已知其偏导数的子表达式：

$$\begin{aligned}\frac{\partial \mathbf{u}}{\partial \mathbf{w}} &= \frac{\partial}{\partial \mathbf{w}}(\mathbf{w} \otimes \mathbf{x}) = diag(\mathbf{x}) \\ \frac{\partial y}{\partial \mathbf{u}} &= \frac{\partial}{\partial \mathbf{u}} sum(\mathbf{u}) = \vec{1}^T\end{aligned}$$

向量链式法则要求将这些偏导数相乘：

$$\frac{\partial y}{\partial \mathbf{w}} = \frac{\partial y}{\partial \mathbf{u}} \frac{\partial \mathbf{u}}{\partial \mathbf{w}} = \vec{1}^T diag(\mathbf{x}) = \mathbf{x}^T$$

为了检验我们的结果，我们可以把点积化简为一个纯粹的标量函数：

$$\begin{aligned}y &= \mathbf{w} \cdot \mathbf{x} = \sum_i^n (w_i x_i) \\ \frac{\partial y}{\partial w_j} &= \frac{\partial}{\partial w_j} \sum_i (w_i x_i) = \sum_i \frac{\partial}{\partial w_j} (w_i x_i) = \frac{\partial}{\partial w_j} (w_j x_j) = x_j\end{aligned}$$

于是：

$$\frac{\partial y}{\partial \mathbf{w}} = [x_1, \dots, x_n] = \mathbf{x}^T$$

太好了！我们的标量结果与向量链式法则的结果一致。

现在，令 $y = \mathbf{w} \cdot \mathbf{x} + b$ ，即 max 激活函数调用中的完整表达式。我们需要计算两个不同的偏导数，但不需要链式法则：

$$\begin{aligned}\frac{\partial y}{\partial \mathbf{w}} &= \frac{\partial}{\partial \mathbf{w}} \mathbf{w} \cdot \mathbf{x} + \frac{\partial}{\partial \mathbf{w}} b = \mathbf{x}^T + \vec{0}^T = \mathbf{x}^T \\ \frac{\partial y}{\partial b} &= \frac{\partial}{\partial b} \mathbf{w} \cdot \mathbf{x} + \frac{\partial}{\partial b} b = 0 + 1 = 1\end{aligned}$$

现在来处理神经元激活 $max(0, \mathbf{w} \cdot \mathbf{x} + b)$ 的偏导数。对标量 z 使用 $max(0, z)$ 函数调用，只是把所有负的 z 值都当作 0。 max 函数的导数是一个分段函数。当 $z \leq 0$ 时，导数为 0，因为此时 z 是常数。当 $z > 0$ 时， max 函数的导数就是 z 的导数，即 1：

$$\frac{\partial}{\partial z} max(0, z) = \begin{cases} 0 & z \leq 0 \\ \frac{dz}{dz} = 1 & z > 0 \end{cases}$$

顺便说明一下函数对标量的广播。当 max 的一个或两个参数为向量时，例如 $max(0, \mathbf{x})$ ，我们把单变量函数 max 广播到各元素上。这是逐元素一元算子的一个例子。为了明确起见：

$$max(0, \mathbf{x}) = \begin{bmatrix} max(0, x_1) \\ max(0, x_2) \\ \dots \\ max(0, x_n) \end{bmatrix}$$

那么，对于广播版本的导数，我们得到一个由 0 和 1 组成的向量，其中：

$$\frac{\partial}{\partial x_i} \max(0, x_i) = \begin{cases} 0 & x_i \leq 0 \\ \frac{dx_i}{dx_i} = 1 & x_i > 0 \end{cases}$$

$$\frac{\partial}{\partial \mathbf{x}} \max(0, \mathbf{x}) = \begin{bmatrix} \frac{\partial}{\partial x_1} \max(0, x_1) \\ \frac{\partial}{\partial x_2} \max(0, x_2) \\ \dots \\ \frac{\partial}{\partial x_n} \max(0, x_n) \end{bmatrix}$$

为了得到 $\text{activation}(\mathbf{x})$ 函数的导数，由于存在嵌套子表达式 $\mathbf{w} \cdot \mathbf{x} + b$ ，我们需要链式法则。按照我们的流程，引入中间标量变量 z 来表示仿射函数，得到：

$$z(\mathbf{w}, b, \mathbf{x}) = \mathbf{w} \cdot \mathbf{x} + b$$

$$\text{activation}(z) = \max(0, z)$$

向量链式法则告诉我们：

$$\frac{\partial \text{activation}}{\partial \mathbf{w}} = \frac{\partial \text{activation}}{\partial z} \frac{\partial z}{\partial \mathbf{w}}$$

我们可以将其改写如下：

$$\frac{\partial \text{activation}}{\partial \mathbf{w}} = \begin{cases} 0 \frac{\partial z}{\partial \mathbf{w}} = \vec{0}^T & z \leq 0 \\ 1 \frac{\partial z}{\partial \mathbf{w}} = \frac{\partial z}{\partial \mathbf{w}} = \mathbf{x}^T & z > 0 \end{cases} \quad (\text{we computed } \frac{\partial z}{\partial \mathbf{w}} = \mathbf{x}^T \text{ previously})$$

然后把 $z = \mathbf{w} \cdot \mathbf{x} + b$ 代回去：

$$\frac{\partial \text{activation}}{\partial \mathbf{w}} = \begin{cases} \vec{0}^T & \mathbf{w} \cdot \mathbf{x} + b \leq 0 \\ \mathbf{x}^T & \mathbf{w} \cdot \mathbf{x} + b > 0 \end{cases}$$

该方程符合我们的直觉。当激活函数把仿射函数的输出 z 截断为 0 时，对任意权重 w_i 的导数都为零。当 $z > 0$ 时，就好像 $\max$ 函数消失了，我们得到的只是 z 对权重的导数。

现在转而考虑神经元激活对 b 的导数，我们得到：

$$\frac{\partial \text{activation}}{\partial b} = \begin{cases} 0 \frac{\partial z}{\partial b} = 0 & \mathbf{w} \cdot \mathbf{x} + b \leq 0 \\ 1 \frac{\partial z}{\partial b} = 1 & \mathbf{w} \cdot \mathbf{x} + b > 0 \end{cases}$$

现在让我们利用这些偏导数来处理整个损失函数。

6 神经网络损失函数的梯度

训练一个神经元需要我们求出损失函数（或称“代价”函数）对模型参数 $\mathbf{w}$ 和 b 的导数。由于我们使用多个向量输入（例如多张图像）和标量目标（例如每张图像一个分类）进行训练，我们需要一些额外的记号。令

$$X = [\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N]^T$$

其中 $N = |X|$ ，然后令

$$\mathbf{y} = [target(\mathbf{x}_1), target(\mathbf{x}_2), \dots, target(\mathbf{x}_N)]^T$$

其中 y_i 是标量。于是代价方程变为：

$$C(\mathbf{w}, b, X, \mathbf{y}) = \frac{1}{N} \sum_{i=1}^N (y_i - activation(\mathbf{x}_i))^2 = \frac{1}{N} \sum_{i=1}^N (y_i - max(0, \mathbf{w} \cdot \mathbf{x}_i + b))^2$$

按照我们的链式法则流程，引入以下中间变量：

$$\begin{aligned} u(\mathbf{w}, b, \mathbf{x}) &= max(0, \mathbf{w} \cdot \mathbf{x} + b) \\ v(y, u) &= y - u \\ C(v) &= \frac{1}{N} \sum_{i=1}^N v^2 \end{aligned}$$

我们先计算对 $\mathbf{w}$ 的梯度。

6.1 对权重的梯度

根据前面的结果，我们知道：

$$\frac{\partial}{\partial \mathbf{w}} u(\mathbf{w}, b, \mathbf{x}) = \begin{cases} \vec{0}^T & \mathbf{w} \cdot \mathbf{x} + b \leq 0 \\ \mathbf{x}^T & \mathbf{w} \cdot \mathbf{x} + b > 0 \end{cases}$$

以及

$$\frac{\partial v(y, u)}{\partial \mathbf{w}} = \frac{\partial}{\partial \mathbf{w}} (y - u) = \vec{0}^T - \frac{\partial u}{\partial \mathbf{w}} = -\frac{\partial u}{\partial \mathbf{w}} = \begin{cases} \vec{0}^T & \mathbf{w} \cdot \mathbf{x} + b \leq 0 \\ -\mathbf{x}^T & \mathbf{w} \cdot \mathbf{x} + b > 0 \end{cases}$$

那么，对于整体梯度，我们得到：

$$\frac{\partial C(v)}{\partial \mathbf{w}} = \frac{\partial}{\partial \mathbf{w}} \frac{1}{N} \sum_{i=1}^N v^2$$

$$\begin{aligned}
&= \frac{1}{N} \sum_{i=1}^N \frac{\partial}{\partial \mathbf{w}} v^2 \\
&= \frac{1}{N} \sum_{i=1}^N \frac{\partial v^2}{\partial v} \frac{\partial v}{\partial \mathbf{w}} \\
&= \frac{1}{N} \sum_{i=1}^N 2v \frac{\partial v}{\partial \mathbf{w}} \\
&= \frac{1}{N} \sum_{i=1}^N \begin{cases} 2v \vec{0}^T = \vec{0}^T & \mathbf{w} \cdot \mathbf{x}_i + b \leq 0 \\ -2v \mathbf{x}_i^T & \mathbf{w} \cdot \mathbf{x}_i + b > 0 \end{cases} \\
&= \frac{1}{N} \sum_{i=1}^N \begin{cases} \vec{0}^T & \mathbf{w} \cdot \mathbf{x}_i + b \leq 0 \\ -2(y_i - u) \mathbf{x}_i^T & \mathbf{w} \cdot \mathbf{x}_i + b > 0 \end{cases} \\
&= \frac{1}{N} \sum_{i=1}^N \begin{cases} \vec{0}^T & \mathbf{w} \cdot \mathbf{x}_i + b \leq 0 \\ -2(y_i - \max(0, \mathbf{w} \cdot \mathbf{x}_i + b)) \mathbf{x}_i^T & \mathbf{w} \cdot \mathbf{x}_i + b > 0 \end{cases} \\
&= \frac{1}{N} \sum_{i=1}^N \begin{cases} \vec{0}^T & \mathbf{w} \cdot \mathbf{x}_i + b \leq 0 \\ -2(y_i - (\mathbf{w} \cdot \mathbf{x}_i + b)) \mathbf{x}_i^T & \mathbf{w} \cdot \mathbf{x}_i + b > 0 \end{cases} \\
&= \begin{cases} \vec{0}^T & \mathbf{w} \cdot \mathbf{x}_i + b \leq 0 \\ -\frac{2}{N} \sum_{i=1}^N (y_i - (\mathbf{w} \cdot \mathbf{x}_i + b)) \mathbf{x}_i^T & \mathbf{w} \cdot \mathbf{x}_i + b > 0 \end{cases} \\
&= \begin{cases} \vec{0}^T & \mathbf{w} \cdot \mathbf{x}_i + b \leq 0 \\ \frac{2}{N} \sum_{i=1}^N (\mathbf{w} \cdot \mathbf{x}_i + b - y_i) \mathbf{x}_i^T & \mathbf{w} \cdot \mathbf{x}_i + b > 0 \end{cases}
\end{aligned}$$

为了解释这个方程，我们可以代入误差项 $e_i = \mathbf{w} \cdot \mathbf{x}_i + b - y_i$ ，得到：

$$\frac{\partial C}{\partial \mathbf{w}} = \frac{2}{N} \sum_{i=1}^N e_i \mathbf{x}_i^T \quad (\text{for the nonzero activation case})$$

由此可以看出，这个计算是对 X 中所有 $\mathbf{x}_i$ 的加权平均。权重就是误差项，即每个输入 $\mathbf{x}_i$ 的目标输出与实际神经元输出之差。平均而言，所得的梯度指向代价或损失更高的方向，因为大的 e_i 会凸显其对应的 $\mathbf{x}_i$ 。设想我们只有一个输入向量， $N = |X| = 1$ ，那么梯度就是 $2e_1 \mathbf{x}_1^T$ 。如果误差为 0，梯度就为零，我们就到达了最小损失。如果 e_1 是较小的正差值，梯度就是沿 $\mathbf{x}_1$ 方向的一小步。如果 e_1 很大，梯度就是沿该方向的一大步。如果 e_1 为负，梯度就会反向，这意味着最高代价在负方向上。

当然，我们希望减小而不是增大损失，这正是[梯度下降](#)递推关系取梯度的负值来更新当前位置的原

因（对于标量学习率 η ）：

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \frac{\partial C}{\partial \mathbf{w}}$$

因为梯度指示的是代价更高的方向，所以我们在相反方向上更新 $\mathbf{x}$ 。

6.2 对偏置的导数

为了优化偏置 b ，我们还需要关于 b 的偏导数。这里再次给出中间变量：

$$\begin{aligned} u(\mathbf{w}, b, \mathbf{x}) &= \max(0, \mathbf{w} \cdot \mathbf{x} + b) \\ v(y, u) &= y - u \\ C(v) &= \frac{1}{N} \sum_{i=1}^N v^2 \end{aligned}$$

我们之前已经计算过方程 $u(\mathbf{w}, b, \mathbf{x})$ 关于偏置的偏导数：

$$\frac{\partial u}{\partial b} = \begin{cases} 0 & \mathbf{w} \cdot \mathbf{x} + b \leq 0 \\ 1 & \mathbf{w} \cdot \mathbf{x} + b > 0 \end{cases}$$

对于 v ，偏导数为：

$$\frac{\partial v(y, u)}{\partial b} = \frac{\partial}{\partial b}(y - u) = 0 - \frac{\partial u}{\partial b} = -\frac{\partial u}{\partial b} = \begin{cases} 0 & \mathbf{w} \cdot \mathbf{x} + b \leq 0 \\ -1 & \mathbf{w} \cdot \mathbf{x} + b > 0 \end{cases}$$

而对于代价函数本身的偏导数，我们得到：

$$\begin{aligned} \frac{\partial C(v)}{\partial b} &= \frac{\partial}{\partial b} \frac{1}{N} \sum_{i=1}^N v^2 \\ &= \frac{1}{N} \sum_{i=1}^N \frac{\partial}{\partial b} v^2 \\ &= \frac{1}{N} \sum_{i=1}^N \frac{\partial v^2}{\partial v} \frac{\partial v}{\partial b} \\ &= \frac{1}{N} \sum_{i=1}^N 2v \frac{\partial v}{\partial b} \\ &= \frac{1}{N} \sum_{i=1}^N \begin{cases} 0 & \mathbf{w} \cdot \mathbf{x} + b \leq 0 \\ -2v & \mathbf{w} \cdot \mathbf{x} + b > 0 \end{cases} \end{aligned}$$

$$\begin{aligned}
&= \frac{1}{N} \sum_{i=1}^N \begin{cases} 0 & \mathbf{w} \cdot \mathbf{x} + b \leq 0 \\ -2(y_i - \max(0, \mathbf{w} \cdot \mathbf{x}_i + b)) & \mathbf{w} \cdot \mathbf{x} + b > 0 \end{cases} \\
&= \frac{1}{N} \sum_{i=1}^N \begin{cases} 0 & \mathbf{w} \cdot \mathbf{x} + b \leq 0 \\ 2(\mathbf{w} \cdot \mathbf{x}_i + b - y_i) & \mathbf{w} \cdot \mathbf{x} + b > 0 \end{cases} \\
&= \begin{cases} 0 & \mathbf{w} \cdot \mathbf{x}_i + b \leq 0 \\ \frac{2}{N} \sum_{i=1}^N (\mathbf{w} \cdot \mathbf{x}_i + b - y_i) & \mathbf{w} \cdot \mathbf{x}_i + b > 0 \end{cases}
\end{aligned}$$

和之前一样，我们可以代入误差项：

$$\frac{\partial C}{\partial b} = \frac{2}{N} \sum_{i=1}^N e_i \quad (\text{for the nonzero activation case})$$

于是偏导数就只是平均误差或零，具体取决于激活水平。为了更新神经元偏置，我们朝代价增大的相反方向微调它：

$$b_{t+1} = b_t - \eta \frac{\partial C}{\partial b}$$

在实践中，把 $\mathbf{w}$ 和 b 合并成单一的向量参数会很方便，而不必处理两个不同的偏导数： $\hat{\mathbf{w}} = [\mathbf{w}^T, b]^T$ 。这也需要对输入向量 $\mathbf{x}$ 略作调整，但会简化激活函数。通过在 $\mathbf{x}$ 的末尾附加一个 1，即 $\hat{\mathbf{x}} = [\mathbf{x}^T, 1]$ ， $\mathbf{w} \cdot \mathbf{x} + b$ 就变为 $\hat{\mathbf{w}} \cdot \hat{\mathbf{x}}$ 。

至此，神经网络损失函数的优化就完成了，因为我们拥有了执行梯度下降所需的两个偏导数。

7 总结

希望你已经一路读到了这里。你正走在理解矩阵微积分的路上！我们在下一节中提供了一个参考文献，总结了本文中的所有规则。也请查看下面带注释的资源链接。

你的下一步是学习矩阵（而不仅仅是向量）的偏导数。例如，你可以查看[矩阵微积分](#)的矩阵微分部分。

致谢。我们感谢 [Yannet Interian](#)（旧金山大学 MS 数据科学项目教师）和 [David Uminsky](#)（MS 数据科学教师/主任）对本文所呈现记号提供的帮助。

8 矩阵微积分参考

8.1 梯度与雅可比矩阵

两个变量的函数的梯度是一个水平 2-向量：

$$\nabla f(x, y) = \left[\frac{\partial f(x, y)}{\partial x}, \frac{\partial f(x, y)}{\partial y} \right]$$

以向量为自变量的向量值函数的雅可比矩阵是一个 $m \times n$ ($m = |\mathbf{f}|$, $n = |\mathbf{x}|$) 矩阵，包含所有可能的标量偏导数：

$$\frac{\partial \mathbf{y}}{\partial \mathbf{x}} = \begin{bmatrix} \nabla f_1(\mathbf{x}) \\ \nabla f_2(\mathbf{x}) \\ \dots \\ \nabla f_m(\mathbf{x}) \end{bmatrix} = \begin{bmatrix} \frac{\partial}{\partial \mathbf{x}} f_1(\mathbf{x}) \\ \frac{\partial}{\partial \mathbf{x}} f_2(\mathbf{x}) \\ \dots \\ \frac{\partial}{\partial \mathbf{x}} f_m(\mathbf{x}) \end{bmatrix} = \begin{bmatrix} \frac{\partial}{\partial x_1} f_1(\mathbf{x}) & \frac{\partial}{\partial x_2} f_1(\mathbf{x}) & \dots & \frac{\partial}{\partial x_n} f_1(\mathbf{x}) \\ \frac{\partial}{\partial x_1} f_2(\mathbf{x}) & \frac{\partial}{\partial x_2} f_2(\mathbf{x}) & \dots & \frac{\partial}{\partial x_n} f_2(\mathbf{x}) \\ \dots & \dots & \dots & \dots \\ \frac{\partial}{\partial x_1} f_m(\mathbf{x}) & \frac{\partial}{\partial x_2} f_m(\mathbf{x}) & \dots & \frac{\partial}{\partial x_n} f_m(\mathbf{x}) \end{bmatrix}$$

恒等函数 $\mathbf{f}(\mathbf{x}) = \mathbf{x}$ 的雅可比矩阵是 I 。

8.2 向量上的逐元素运算

使用算子 $\circ$ (例如 $+$) 在向量 $\mathbf{w}$ 和 $\mathbf{x}$ 上定义通用逐元素运算：

$$\begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix} = \begin{bmatrix} f_1(\mathbf{w}) \circ g_1(\mathbf{x}) \\ f_n(\mathbf{w}) \circ g_2(\mathbf{x}) \\ \vdots \\ f_n(\mathbf{w}) \circ g_n(\mathbf{x}) \end{bmatrix}$$

关于 $\mathbf{w}$ 的雅可比矩阵 ($\mathbf{x}$ 类似) 为：

$$J_{\mathbf{w}} = \frac{\partial \mathbf{y}}{\partial \mathbf{w}} = \begin{bmatrix} \frac{\partial}{\partial w_1} (f_1(\mathbf{w}) \circ g_1(\mathbf{x})) & \frac{\partial}{\partial w_2} (f_1(\mathbf{w}) \circ g_1(\mathbf{x})) & \dots & \frac{\partial}{\partial w_n} (f_1(\mathbf{w}) \circ g_1(\mathbf{x})) \\ \frac{\partial}{\partial w_1} (f_2(\mathbf{w}) \circ g_2(\mathbf{x})) & \frac{\partial}{\partial w_2} (f_2(\mathbf{w}) \circ g_2(\mathbf{x})) & \dots & \frac{\partial}{\partial w_n} (f_2(\mathbf{w}) \circ g_2(\mathbf{x})) \\ \dots & \dots & \dots & \dots \\ \frac{\partial}{\partial w_1} (f_n(\mathbf{w}) \circ g_n(\mathbf{x})) & \frac{\partial}{\partial w_2} (f_n(\mathbf{w}) \circ g_n(\mathbf{x})) & \dots & \frac{\partial}{\partial w_n} (f_n(\mathbf{w}) \circ g_n(\mathbf{x})) \end{bmatrix}$$

在约束 (逐元素对角条件) 下，即 $f_i(\mathbf{w})$ 和 $g_i(\mathbf{x})$ 分别最多访问 w_i 和 x_i ，雅可比矩阵简化为对角矩阵：

$$\frac{\partial \mathbf{y}}{\partial \mathbf{w}} = \text{diag} \left(\frac{\partial}{\partial w_1} (f_1(w_1) \circ g_1(x_1)), \frac{\partial}{\partial w_2} (f_2(w_2) \circ g_2(x_2)), \dots, \frac{\partial}{\partial w_n} (f_n(w_n) \circ g_n(x_n)) \right)$$

以下是一些逐元素算子示例：

Op	Partial with respect to $\mathbf{w}$	Partial with respect to $\mathbf{x}$
+	$\frac{\partial(\mathbf{w}+\mathbf{x})}{\partial \mathbf{w}} = I$	$\frac{\partial(\mathbf{w}+\mathbf{x})}{\partial \mathbf{x}} = I$
-	$\frac{\partial(\mathbf{w}-\mathbf{x})}{\partial \mathbf{w}} = I$	$\frac{\partial(\mathbf{w}-\mathbf{x})}{\partial \mathbf{x}} = -I$
$\otimes$	$\frac{\partial(\mathbf{w} \otimes \mathbf{x})}{\partial \mathbf{w}} = \text{diag}(\mathbf{x})$	$\frac{\partial(\mathbf{w} \otimes \mathbf{x})}{\partial \mathbf{x}} = \text{diag}(\mathbf{w})$
$\oslash$	$\frac{\partial(\mathbf{w} \oslash \mathbf{x})}{\partial \mathbf{w}} = \text{diag}(\dots \frac{1}{x_i} \dots)$	$\frac{\partial(\mathbf{w} \oslash \mathbf{x})}{\partial \mathbf{x}} = \text{diag}(\dots \frac{-w_i}{x_i^2} \dots)$

8.3 标量扩展

将标量 z 加到向量 $\mathbf{x}$ 上， $\mathbf{y} = \mathbf{x} + z$ 实际上就是 $\mathbf{y} = \mathbf{f}(\mathbf{x}) + \mathbf{g}(z)$ ，其中 $\mathbf{f}(\mathbf{x}) = \mathbf{x}$ ， $\mathbf{g}(z) = \vec{1}z$ 。

$$\frac{\partial}{\partial \mathbf{x}}(\mathbf{x} + z) = \text{diag}(\vec{1}) = I$$

$$\frac{\partial}{\partial z}(\mathbf{x} + z) = \vec{1}$$

标量乘法得到：

$$\frac{\partial}{\partial \mathbf{x}}(\mathbf{x}z) = Iz$$

$$\frac{\partial}{\partial z}(\mathbf{x}z) = \mathbf{x}$$

8.4 向量归约

向量和关于其中一个向量的偏导数为：

$$\nabla_{\mathbf{x}} y = \frac{\partial y}{\partial \mathbf{x}} = \left[\frac{\partial y}{\partial x_1}, \frac{\partial y}{\partial x_2}, \dots, \frac{\partial y}{\partial x_n} \right] = \left[\sum_i \frac{\partial f_i(\mathbf{x})}{\partial x_1}, \sum_i \frac{\partial f_i(\mathbf{x})}{\partial x_2}, \dots, \sum_i \frac{\partial f_i(\mathbf{x})}{\partial x_n} \right]$$

对于 $y = \text{sum}(\mathbf{x})$ ：

$$\nabla_{\mathbf{x}} y = \vec{1}^T$$

对于 $y = \text{sum}(\mathbf{x}z)$ 且 $n = |x|$ ，我们得到：

$$\nabla_{\mathbf{x}} y = [z, z, \dots, z]$$

$$\nabla_z y = \text{sum}(\mathbf{x})$$

向量点积 $y = \mathbf{f}(\mathbf{w}) \cdot \mathbf{g}(\mathbf{x}) = \sum_i^n (w_i x_i) = \text{sum}(\mathbf{w} \otimes \mathbf{x})$ 。代入 $\mathbf{u} = \mathbf{w} \otimes \mathbf{x}$ 并利用向量链式法则，我们得到：

$$\frac{d\mathbf{u}}{d\mathbf{x}} = \frac{d}{d\mathbf{x}}(\mathbf{w} \otimes \mathbf{x}) = \text{diag}(\mathbf{w})$$

$$\frac{dy}{d\mathbf{u}} = \frac{d}{d\mathbf{u}} \text{sum}(\mathbf{u}) = \vec{1}^T$$

$$\frac{dy}{d\mathbf{x}} = \frac{dy}{d\mathbf{u}} \times \frac{d\mathbf{u}}{d\mathbf{x}} = \vec{1}^T \times \text{diag}(\mathbf{w}) = \mathbf{w}^T$$

类似地， $\frac{dy}{d\mathbf{w}} = \mathbf{x}^T$ 。

8.5 链式法则

向量链式法则是通用形式，其他法则均可由其退化得到。当 f 是单变量 x 的函数且所有中间变量 u 都是单变量的函数时，适用单变量链式法则。当部分或全部中间变量是多个变量的函数时，适用单变量全导数链式法则。在所有其他情况下，适用向量链式法则。

单变量规则 **单变量全导数规则** **向量规则**

$$\frac{df}{dx} = \frac{df}{du} \frac{du}{dx} \quad \frac{\partial f(u_1, \dots, u_n)}{\partial x} = \frac{\partial f}{\partial \mathbf{u}} \frac{\partial \mathbf{u}}{\partial x} \quad \frac{\partial}{\partial \mathbf{x}} \mathbf{f}(\mathbf{g}(\mathbf{x})) = \frac{\partial \mathbf{f}}{\partial \mathbf{g}} \frac{\partial \mathbf{g}}{\partial \mathbf{x}}$$

9 记号

粗体小写字母（如 $\mathbf{x}$ ）表示向量，斜体字母（如 x ）表示标量。 x_i 是向量 $\mathbf{x}$ 的第 i^{th} 个元素，用斜体表示，因为单个向量元素是标量。 $|\mathbf{x}|$ 表示“向量 $\mathbf{x}$ 的长度”。

$\mathbf{x}^T$ 的上标 T 表示所指向量的转置。

$\sum_{i=a}^b x_i$ 就是一个 for 循环，从 a 到 b 迭代 i ，并累加所有的 x_i 。

记号 $f(x)$ 指一个名为 f 、自变量为 x 的函数。

I 表示适当维数的方阵“单位矩阵”，除对角线全为 1 外，其余各处全为 0。

$diag(\mathbf{x})$ 构造一个对角元素取自向量 $\mathbf{x}$ 的矩阵。

点积 $\mathbf{w} \cdot \mathbf{x}$ 是各元素逐元素相乘后的求和： $\sum_i^n (w_i x_i) = sum(\mathbf{w} \otimes \mathbf{x})$ 。或者，你也可以把它看作 $w^T x$ 。

微分 $\frac{d}{dx}$ 是一个把单参数函数映射为另一个函数的算子。也就是说， $\frac{d}{dx} f(x)$ 把 $f(x)$ 映射为它关于 x 的导数，这与 $\frac{df(x)}{dx}$ 是同一回事。此外，如果 $y = f(x)$ ，那么 $\frac{dy}{dx} = \frac{df(x)}{dx} = \frac{d}{dx} f(x)$ 。

函数关于 x 的偏导数 $\frac{\partial}{\partial x} f(x)$ ，在保持所有其他变量不变的情况下执行通常的标量求导。

f 关于向量 $\mathbf{x}$ 的梯度 $\nabla f(\mathbf{x})$ ，将某个特定标量函数的所有偏导数组织起来。

雅可比矩阵通过堆叠的方式将多个函数的梯度组织成一个矩阵：

$$J = \begin{bmatrix} \nabla f_1(\mathbf{x}) \\ \nabla f_2(\mathbf{x}) \end{bmatrix}$$

以下记号表示：在 $condition_1$ 时 y 取值为 a ，在 $condition_2$ 时取值为 b 。

$$y = \begin{cases} a & condition_1 \\ b & condition_2 \end{cases}$$

10 资源

[Wolfram Alpha](#) 可以进行符号矩阵代数运算，此外还有一个很酷的专门[矩阵微积分求导器](#)。

在网上查找资源时，请搜索“矩阵微积分”而不是“向量微积分”。以下是对 [Google 搜索](#) 结果中排名靠前的链接的一些评论：

- https://en.wikipedia.org/wiki/Matrix_calculus 维基百科的词条其实相当不错，而且对不同布局惯例有很好的描述。回顾一下，我们使用分子布局，即在雅可比矩阵中变量水平排列、函数垂直排列。维基百科对[全导数](#)也有很好的描述，但要注意他们使用的记号与我们略有不同。我们始终使用 ∂x 记号，而不是 dx 。
- <http://www.ee.ic.ac.uk/hp/staff/dmb/matrix/calculus.html> 这个页面有关于矩阵微分的一节，包含一些有用的恒等式；该作者使用分子布局。读完本文后，从这里开始学习矩阵微分与向量微分的区别，可能是个不错的起点。
- <https://www.colorado.edu/engineering/CAS/courses.d/IFEM.d/IFEM.AppC.d/IFEM.AppC.pdf> 这是“有限元方法导论”课程笔记的一部分，我相信作者是 [Carlos A. Felippa](#)。他的雅可比矩阵相对我们的记号是转置的，因为他使用分母布局。
- <http://www.ee.ic.ac.uk/hp/staff/dmb/matrix/calculus.html> 这个页面为各种向量和矩阵计算了大量有用的导数。是一份很棒的速查表。基本没有讲解，只是一组规则。
- <https://www.math.uwaterloo.ca/~hwolkowi/matrixcookbook.pdf> 另一份速查表，总体上侧重于矩阵运算，且比前一条有更多的讲解。
- <https://www.comp.nus.edu.sg/~cs5240/lecture/matrix-differentiation.pdf> 一组有用的幻灯片。

要了解更多关于神经网络以及优化和反向传播背后的数学，我们强烈推荐 [Michael Nielsen](#) 的著作。

对于特别关注卷积神经网络的人，请查看[用于深度学习的卷积算术指南](#)。

我们引用了[全导数](#)法则，这是一个重要的概念，它仅仅意味着关于 x 的导数必须考虑所有作为 x 之函数的变量关于 x 的导数。